



*Asociación Profesional de Cuerpos Superiores
de Sistemas y Tecnologías de la Información
de las Administraciones Públicas.*

Temas Específicos para la preparación de la Oposición al Cuerpo
Superior de Sistemas y Tecnologías de la Información de la
Administración del Estado.

TEMAS ESPECÍFICOS I: Organización y gestión de los sistemas de información

**Tema 37. Metodologías ágiles para la gestión de proyectos.
Metodologías lean**

AUTOR: Miguel Rodríguez Martos

Actualización 2017

ÍNDICE

1	INTRODUCCIÓN	4
2	FUNDAMENTOS DE LA GESTIÓN ÁGIL DE PROYECTOS .	6
2.1	MANIFIESTO ÁGIL	7
2.2	¿CUÁNDO GESTIONAR DE FORMA ÁGIL UN PROYECTO?	8
2.3	METODOLOGÍAS ÁGILES MÁS POPULARES	9
2.3.1	SCRUM	9
2.3.2	KANBAN	9
2.3.3	EXTREME PROGRAMMING (XP)	10
2.3.4	FEATURE DRIVEN DEVELOPMENT (FDD)	10
2.3.5	TEST DRIVEN DEVELOPMENT (TDD).....	11
2.3.6	BEHAVIOR DRIVEN DEVELOPMENT (BDD)	11
2.3.7	DYNAMIC SYSTEMS DEVELOPMENT METHOD (DSDM).....	11
2.3.8	CRYSTAL	12
2.3.9	LEAN.....	12
2.4	SITUACIÓN DE AGILE EN LAS ORGANIZACIONES	12
2.4.1	SITUACIÓN EN LA ADMINISTRACIÓN.....	13
3	CICLO DE VIDA DE UN PROYECTO ÁGIL	14
3.1	INICIANDO EL PROYECTO: CONCEBIR	15
3.1.1	EL EQUIPO	15
3.1.2	DURACIÓN DE LAS ITERACIONES	16
3.1.3	HERRAMIENTAS DE GESTIÓN DE PROYECTOS ÁGILES.....	18
3.2	PLANIFICANDO EL TRABAJO: ESPECULAR	19
3.2.1	REGISTRO DE REQUISITOS	20
3.2.2	DISTRIBUCIÓN DE FUNCIONALIDADES	20
3.3	CONSTRUYENDO EL PRODUCTO: EXPLORAR.....	21
3.3.1	CONTROL DEL AVANCE	22
3.4	REVISANDO EL PROGRESO: ADAPTAR.....	24
3.4.1	SOLUCIÓN DE PROBLEMAS	24
3.5	FINALIZANDO EL PROYECTO: CERRAR	25
4	AGILE A GRAN ESCALA.....	26
4.1	COORDINACIÓN.....	26
5	DEVOPS	29
5.1	VALORES Y PRINCIPIOS.....	29
5.2	CICLO	30
6	LEAN	31
6.1	PRINCIPIOS FUNDAMENTALES.....	31

6.2	TÉCNICAS Y PRÁCTICAS	33
6.2.1	VALUE STREAM MAPPING (VSM)	33
6.2.2	5S	34
6.2.3	KAIZEN	34
6.2.4	AUTOMATIZACIÓN	35
6.2.5	SISTEMA KANBAN	35
6.2.6	POKA-YOKE	35
6.2.7	JIDOKA	36
6.2.8	QUICK CHANGEOVER	36
6.2.9	SIX SIGMA	36

1 Introducción

Algunos gestores de proyectos tienden a profesionalizarse en la aplicación de una estrategia de gestión de proyectos y la aplican con independencia del tipo de proyecto y de las necesidades del mismo. La mayor parte de los gestores tienen una gran experiencia con metodologías tradicionales (en cascada) y por el contrario un conocimiento menos maduro de las adaptativas (ágiles). No obstante, en los últimos años se ha visto en el desarrollo de software un impulso importante en favor de las metodologías ágiles sobre las metodologías predictivas más tradicionales, impulso soportado por las ventajas que las primeras aportan a la gestión de proyectos.

La realidad es que no existe una metodología que sea mejor para *todos* los proyectos, sino que según sus características y el contexto donde se desarrollen, una u otra metodología puede ser más favorable para finalizar el proyecto con éxito, entendiendo como tal la satisfacción de las necesidades del cliente entregando el resultado a tiempo y con un presupuesto limitado.

Por tanto, es necesario conocer los fundamentos y procedimientos a seguir en ambos tipos de metodologías para, en primer lugar, valorar su idoneidad, y, en segundo término, gestionar el proyecto de manera apropiada. Sólo conociendo adecuadamente ambas metodologías se podrá escoger con libertad y de forma razonada en función de las necesidades del cliente y del proyecto, y no llevado por preferencias personales.

En este tema se van a presentar las bases y objetivos de las metodologías ágiles, los procedimientos que se siguen durante el ciclo de vida del proyecto y la forma en que se puede gestionar el alcance del mismo para entregar valor al cliente de forma inteligente. También se van a analizar las principales diferencias entre las metodologías predictivas y las ágiles, en forma de claves que ayudarán a determinar la mejor opción cuando haya que enfrentar esta decisión en la piel de un gestor de proyecto.

Todos estos fundamentos y procesos resultarán de aplicación con independencia de la implementación concreta que se escoja entre las distintas alternativas ágiles, si bien la omnipresencia de Scrum en todo tipo de organizaciones y su flexibilidad para incluir procesos de otras variantes ágiles hace que sus formas de aplicación sirvan como marco para explicar la implementación práctica. También se hará un repaso sobre las principales alternativas ágiles destacando los fundamentos de cada una.

Por otra parte, se verá en este tema un concepto que ha emergido en los últimos años para aplicar *agile* a gran escala. Los equipos ágiles han funcionado muy bien en entornos pequeños, para la entrega de un proyecto con equipos de pocas personas localizados en la misma oficina. Pero hoy en día en grandes organizaciones innovadoras también se está aplicando *agile* para gestionar todo un programa o una estrategia, donde se realizan múltiples proyectos interrelacionados, cada uno de ellos realizado de forma ágil y a su vez coordinados en un nivel superior también de forma ágil. En lugar de un equipo ágil, se trabaja en un equipo de equipos ágiles.

Uno de los conceptos más novedosos relacionados con las prácticas ágiles y que está creciendo con rapidez es DevOps, una nueva forma de organizar el ciclo de vida de un producto donde la separación tradicional entre desarrollo y operaciones se pretende diluir. Por su relación con *Lean* y las prácticas ágiles, se dedicará un apartado a este nuevo paradigma.

Finalmente, se dedicará un capítulo a la metodología *Lean*, una filosofía que se adaptó al desarrollo de software y que está presente en las raíces de las metodologías ágiles con las que comparte muchos principios.

En definitiva, después de estudiar este tema el lector será capaz de reconocer los principios comunes y objetivos de las metodologías ágiles, sabrá cómo aplicarlos y sobre todo cómo organizar y gestionar un proyecto ágil. Conocerá de qué manera se puede realizar la gestión de los parámetros fundamentales del proyecto (tiempo, presupuesto, calidad, alcance) y tendrá presentes las principales alternativas de metodologías ágiles y sus puntos clave. También conocerá la forma de aplicar *agile* a gran escala para gestionar grandes programas e iniciativas. Se familiarizará con DevOps y su marco de actuación. Y finalmente entenderá la metodología *Lean*, sus principios y la forma en que se aplica, en forma de técnicas y prácticas que ayudan a la realización de sus objetivos.

Dada la finalidad con la que se realiza este texto, que es la preparación de un proceso selectivo de oposición, resulta conveniente indicar que el interés de este tema no se limita al primer ejercicio de examen de conocimientos, sino también de manera relevante al último ejercicio práctico donde de forma

habitual se requiere planificar un proyecto y tomar y justificar decisiones en torno a la metodología y ciclo de vida más adecuado.

Por último, conviene destacar que este tema mantiene el foco en todo momento en la gestión, organización y control del proyecto, y se aborda desde ese punto de vista transversal y esa visión de alto nivel, como corresponde al bloque temático donde se encuadra. El contenido se complementa con otro tema del bloque III dedicado a metodologías de desarrollo ágiles, que abordará la perspectiva del equipo de desarrollo y explicará el detalle de las principales metodologías como Scrum y Kanban.

2 Fundamentos de la gestión ágil de proyectos

En este apartado se va a realizar una primera aproximación a la gestión de proyectos siguiendo metodologías ágiles, de manera que se obtenga un mejor entendimiento y comprensión de sus fundamentos, principios y objetivos primarios.

En pocas palabras, la gestión ágil de proyectos se podría definir como el proceso por el cual un proyecto puede ser gestionado e implementado en pequeñas partes, aportando valor al cliente con entregas frecuentes. Esas pequeñas partes son lo que se conoce como iteraciones o sprints. Se usarán ambos términos como sinónimos en este texto (sprint es un término asociado a la metodología Scrum, la metodología ágil más popular).

Si se piensa en las metodologías tradicionales orientadas a la planificación, tipo cascada, el proyecto sigue una serie de pasos secuenciales: inicialmente se definen y se cierran todos los requisitos, posteriormente se diseña a distintos niveles el producto que cumple dichos requisitos, a continuación, siguiendo los diseños, se construye ese producto, para luego probarlo y finalmente entregarlo. Si el proyecto dura un año, el cliente no tiene ningún resultado ni aportación a su negocio hasta que ese tiempo finaliza. Métrica es un ejemplo de metodología en cascada, usada en la administración pública.

En los proyectos ágiles la duración de una iteración es muy corta comparada con los parámetros de proyectos tradicionales, típicamente entre 2 y 4 semanas. Cada vez que termina una iteración, el cliente recibe un resultado que le aporta valor.

Pero en *agile* no se trata sólo de entregar en pequeñas iteraciones, supone evolucionar hacia un nuevo modelo de gestión más dinámico, donde el alcance del proyecto ya no es algo cerrado y fijo desde el inicio, sino que es flexible. Se trabaja codo a codo con el cliente para que en el tiempo planificado y con el presupuesto establecido, el equipo entregue los elementos de funcionalidad que aporten más valor. Por el camino se pueden sacar unos elementos y añadir otros, modificarlos, re-priorizar, etc. para asegurar que lo que se entrega es realmente lo que el cliente necesita.

En un proyecto tradicional, el alcance queda cerrado desde el principio. Cuando se detecta una nueva necesidad a mitad de camino (porque las necesidades de negocio evolucionan de forma natural, y más hoy día), hay que seguir el procedimiento de control definido. Esto supone crear un diseño de alto nivel que defina la nueva necesidad, estimar el esfuerzo de forma precisa, conseguir una aprobación, planificar su introducción, conseguir los recursos y entonces comenzar el desarrollo. Este proceso es muchas veces largo y disruptivo. Siguiendo una metodología ágil, se trabaja de forma cercana con el usuario, se definen nuevos requisitos y se añaden a la lista de forma natural, se estima su esfuerzo a alto nivel y se prioriza en función del valor que aporta. Se podría acomodar en la siguiente iteración si así se decide.

En la siguiente tabla se ha tratado de resumir las principales diferencias entre ambos tipos de metodologías, enfrentando los distintos elementos que las componen.

Metodologías tradicionales	Metodologías ágiles
Orientadas a la planificación	Orientadas al aprendizaje
Comunicación con el cliente poco frecuente	Frecuente comunicación con el cliente
Proyectos grandes entregados de una sola vez (típicamente de 9-12 meses)	Proyectos cortos enfocados en las prioridades de negocio (típicamente de 3-6 meses)
Requisitos definidos por adelantado	Requisitos que evolucionan durante el proyecto
Desarrollo en distintas fases con entregables entre ellas	Entregas de código funcionando en iteraciones cortas (típicamente 2-4 semanas)
Desarrollo por capas: presentación, negocio, interfaces, etc.	Desarrollo de funcionalidad de extremo a extremo en iteraciones
La integración de las capas ocurre al final de la construcción	Integración continua

Metodologías tradicionales	Metodologías ágiles
Las pruebas se realizan al final	Las pruebas se realizan durante la iteración, tanto pruebas unitarias como funcionales
Alto coste del cambio	Bajo coste del cambio

Figura 1 Metodologías ágiles vs tradicionales

La firma del manifiesto ágil en 2001 se conoce como el punto de partida del concepto *agile*. Revisar el manifiesto ayuda a entender perfectamente los fundamentos y principios sobre los que se levantan todas las metodologías ágiles.

2.1 Manifiesto ágil

En febrero de 2001, representantes de diversas metodologías de desarrollo de software que de una u otra manera presentaban alternativas a los tradicionales procesos de desarrollo, pesados e intensivos en documentación, se reunieron para buscar puntos y acuerdos comunes entre todas ellas. Lo que surgió fue el Manifiesto Ágil de desarrollo de software.

El manifiesto parte de una serie de afirmaciones basadas en la experiencia de los firmantes, que coinciden en valorar:

- Individuos e interacciones sobre procesos y herramientas
- Software funcionando sobre documentación extensiva
- Colaboración con el cliente sobre negociación contractual
- Respuesta ante el cambio sobre seguir un plan

Esto es, aun valorando los elementos de la derecha, se valoran más los de la izquierda. Estos cuatro valores resumen las ideas que están detrás de las metodologías abanderadas por las personas que acuerdan el manifiesto.

El concepto ágil está basado en modelos organizativos que ponen en primer lugar a las personas, primando valores como la confianza y el respeto mutuo, donde se actúa pensando que las personas son lo más valioso e importante de la organización. Los procesos deben ser una ayuda y un soporte para guiar el trabajo, deben adaptarse a la organización, a los equipos y a las personas, y no al revés. Es más importante un equipo en forma y motivado que los controles de proceso definidos. En esto hace hincapié el primer valor, probablemente el más importante.

En los otros tres se pone de manifiesto que la documentación, la negociación contractual y los planes no son lo primero, sino que hay otros elementos que hay que poner por delante. Pero esto no quiere decir que en un proyecto ágil no se documente, o que no haya contrato, ni planificación. De hecho, se planifica mucho, continuamente, lo que ocurre es que no se pone el plan por encima de la capacidad para hacer cambios, sino que, si surgen cambios en el entorno, se adapta el plan.

De igual manera se prefiere el trabajo entre cliente y equipo como una asociación colaborativa para conseguir un proyecto que es común, y del que todos se corresponsabilizan. No se acude al contrato como si fuera una disputa entre dos partes, pero aun así un contrato es necesario para fijar los términos y condiciones.

Finalmente, la documentación no es tan exhaustiva como en un proyecto tradicional, se realiza con el propósito de reflejar la funcionalidad que se está desarrollando de forma que pueda ser referenciada y se complementa con el uso de herramientas donde se recogen y se describen los requisitos.

De esta manera, las metodologías ágiles siguen los siguientes doce principios, que derivan de los cuatro valores anteriores y que son los que forman propiamente el manifiesto ágil:

1. Nuestra mayor prioridad es satisfacer al cliente mediante la entrega temprana y continua de software con valor.

2. Aceptamos que los requisitos cambien, incluso en etapas tardías del desarrollo. Los procesos Ágiles aprovechan el cambio para proporcionar ventaja competitiva al cliente.
3. Entregamos software funcional frecuentemente, entre dos semanas y dos meses, con preferencia al periodo de tiempo más corto posible.
4. Los responsables de negocio y los desarrolladores trabajamos juntos de forma cotidiana durante todo el proyecto.
5. Los proyectos se desarrollan en torno a individuos motivados. Hay que darles el entorno y el apoyo que necesitan, y confiarles la ejecución del trabajo.
6. El método más eficiente y efectivo de comunicar información al equipo de desarrollo y entre sus miembros es la conversación cara a cara.
7. El software funcionando es la medida principal de progreso.
8. Los procesos ágiles promueven el desarrollo sostenible. Los promotores, desarrolladores y usuarios debemos ser capaces de mantener un ritmo constante de forma indefinida.
9. La atención continua a la excelencia técnica y al buen diseño mejora la agilidad.
10. La simplicidad, o el arte de maximizar la cantidad de trabajo no realizado, es esencial.
11. Las mejores arquitecturas, requisitos y diseños emergen de equipos auto-organizados.
12. A intervalos regulares el equipo reflexiona sobre cómo ser más efectivo para la continuación ajustar y perfeccionar su comportamiento en consecuencia.

2.2 ¿Cuándo gestionar de forma ágil un proyecto?

Las metodologías tradicionales de gestión de proyectos funcionan, son efectivas, ayudan a alcanzar los objetivos y de hecho son la elección más acertada en determinados contextos de aplicación. En muchos casos tomar la decisión de aplicar una forma de gestión u otra resulta más claro, pero en otros, ambas opciones son posibles, incluso combinaciones intermedias que adoptan características de una y otra. En estos casos *grises*, la elección final dependerá de la preparación y experiencia del gestor y de su equipo.

En este punto se va a tratar de definir cuáles son los proyectos y entornos que pueden verse especialmente favorecidos por la aplicación metodologías ágiles, y qué características por el contrario aconsejarían seguir una metodología tradicional.

Los proyectos que serían grandes candidatos a ser realizados de forma ágil tendrán varias de las siguientes características:

- Los requisitos no están muy claros o se espera que cambien por necesidades del negocio u otros motivos.
- El cliente no conoce la nueva tecnología y requiere ver las posibilidades que ofrece para poder formular requisitos completos y definidos.
- Se desea un producto de calidad, pero no necesariamente completo desde el inicio.
- El cliente está involucrado y puede dedicar tiempo al proyecto.
- Se requieren resultados rápidos.
- Existe un ambiente de colaboración y confianza entre las partes.
- El producto o sistema se puede desarrollar de forma iterativa, no requiere una implementación única.
- Se pueden obtener resultados pronto, que aporten valor al cliente.

Sin embargo, si el proyecto a afrontar consiste en realizar algo que se ha hecho anteriormente, precisa de una serie de pasos y procesos bien definidos, el alcance está bien delimitado y los requisitos no son cambiantes, lo más adecuado sería realizar el proyecto en cascada. De igual manera un proyecto donde el producto no se pudiera entregar de forma incremental tampoco se realizaría adecuadamente de

forma ágil (piénsese por ejemplo en una interfaz con otro sistema, que sólo funcionará si está completa al 100%). También es frecuente encontrar organizaciones que por motivos culturales, históricos o políticos no están bien preparadas para acoger las metodologías ágiles. La disposición de la organización es algo que por tanto también debe evaluarse.

Entre medias hay muchos escenarios donde el gestor deberá decidir qué beneficios priman sobre otros para elegir la mejor manera de realizar el proyecto. También son posibles métodos híbridos, donde haya partes del proyecto realizadas de forma tradicional y otras de forma ágil. En estos casos habrá que prestar especial atención a la coordinación entre ambas líneas y a los hitos o puntos de encuentro donde ambas entreguen resultados.

Por último, también es posible adoptar a una misma iniciativa características de una y otra metodología. Siempre habrá una que primará, que conformará la base principal, a la que se añadirán procesos de la otra que resulten especialmente convenientes. Por ejemplo, a un proyecto tradicional se le puede añadir al inicio una pequeña fase realizada de forma ágil, con el objetivo de que el cliente se familiarice con la aplicación y formule adecuadamente los requisitos que posteriormente se desarrollarán en un proyecto en cascada. O bien a un proyecto desarrollado de forma ágil, se le pueden añadir al final etapas de pruebas y validación tradicionales para realizar una única entrega final a producción. Estas formas híbridas requerirán de gran experiencia por parte del gestor para solventar de la forma más eficaz las distintas etapas del proyecto.

2.3 Metodologías ágiles más populares

Aunque este tema se aborda desde la perspectiva de la organización y gestión de proyectos empleando principios y procesos transversales, para conocimiento del lector en este apartado se van a citar las metodologías ágiles más empleadas, junto con una descripción de sus principales características. En el tema dedicado a las metodologías ágiles del bloque de ingeniería de los sistemas de información (bloque III del temario) se detallarán en mayor medida las más relevantes desde el punto de vista de su implementación.

2.3.1 Scrum

Scrum es, con diferencia, la metodología ágil más popular, debido a su simplicidad y su productividad demostrada en multitud de proyectos. También por su capacidad de actuar de continente que pueda recoger características de otras metodologías como Kanban (Scrumban) o XP (Scrum+XP).

En ella, el *product owner* trabaja en coordinación con los usuarios de negocio y con el equipo de desarrollo para crear y priorizar el *product backlog*, una lista de trabajo pendiente que detalla todos los requisitos en el alcance del proyecto. El *backlog* incluye todo el trabajo que se ha de realizar, tanto funcionalidades, como errores a corregir, o requisitos no funcionales, y se documenta en forma de historias de usuario. El equipo desarrolla las funcionalidades en iteraciones cortas llamadas *sprints*, típicamente de entre 2 y 4 semanas. Cada *sprint* tiene un alcance que es una parte del *product backlog*, que se cierra al inicio de cada iteración y que se denomina *sprint backlog*. Una vez que se entrega un *sprint*, el *backlog* del proyecto se analiza y reprioriza si es necesario para definir el siguiente *sprint*.

El *scrum master* es el gestor de la eficiencia del equipo, el encargado de que se sigan de forma adecuada los procesos de Scrum y de liderar la resolución de impedimentos para que el equipo pueda centrarse en su trabajo.

2.3.2 Kanban

Kanban es un método usado para gestionar la creación de productos con especial atención a la entrega continuada y a tiempo, sin sobrecargar al equipo de desarrollo. Ayuda a que los equipos trabajen y colaboren de forma más eficiente. Se basa en tres principios:

- Visualizar lo que se hace

- Limitar la cantidad de trabajo en curso
- Mejorar el flujo de trabajo

Como todas, se basa en el desarrollo incremental del producto y su mayor aportación son las técnicas visuales que emplea para ver el flujo de trabajo. El objetivo de esta visualización es que quede claro el trabajo a realizar, en qué está trabajando cada persona y la prioridad de las tareas.

En Kanban se define por cada fase del ciclo de trabajo (por ejemplo, en fase de pruebas) cuál es el número máximo de tareas que pueden realizarse a la vez. No se puede avanzar una tarea a una fase que ya está en su límite de capacidad. Limitar el trabajo en curso en cada fase es una forma de evitar que se empiecen cosas nuevas hasta que se hayan cerrado las que están en curso. La prioridad es por tanto cerrar las tareas comenzadas.

Para mejorar el flujo de trabajo se mide y se trata de optimizar el *lead time*, o tiempo desde que se hace una petición hasta que se entrega. Es lo que espera el cliente hasta que ve su petición implantada.

2.3.3 Extreme Programming (XP)

XP es una metodología que se basa en un enfoque disciplinado para entregar software de calidad rápido y de forma continuada. Promueve una alta involucración del cliente con ciclos rápidos de feedback, la realización continuada de pruebas, planificación constante y trabajo en equipo, de modo que se puedan realizar entregas en ciclos muy cortos, típicamente de una a tres semanas.

El cliente escribe las historias de usuario, las prioriza y decide cuáles se implementan en cada iteración. El programador escribe las pruebas unitarias y produce el código del sistema. El probador escribe junto con el cliente las pruebas funcionales y prueba regularmente. El coach es el responsable de que se siga el proceso global, y el gestor (o big boss) es el punto de enlace entre el cliente y el equipo de desarrollo.

Se basa en cinco principios básicos: simplicidad (para maximizar el valor creado con la mínima inversión), comunicación (diaria y preferiblemente cara a cara), feedback (temprano y periódico), respeto (de forma que todos los miembros del equipo se sientan valorados) y coraje (para ser honestos sobre el progreso del proyecto y adaptarse a los cambios).

Sus características fundamentales incluyen:

- Desarrollo iterativo e incremental
- Pruebas unitarias continuas
- Programación en parejas
- Integración del equipo con el cliente
- Corrección de todos los errores antes de añadir nueva funcionalidad
- Refactorización del código para aumentar su legibilidad y mantenibilidad
- Propiedad repartida del código, todos pueden corregir y extender cualquier parte del código
- Simplicidad en el código

2.3.4 Feature Driven Development (FDD)

FDD basa su proceso en funcionalidades o *features*, que son pequeñas funciones del producto a entregar que son valoradas por el cliente. Se diseña por funcionalidad y se construye por funcionalidad.

El ciclo de FDD consta de los siguientes pasos:

- 1) Desarrollo de un modelo del producto
- 2) Construir una lista de funcionalidades
- 3) Planificación basada en las funcionalidades identificadas

- 4) Diseño basado en las funcionalidades, realizado en iteraciones
- 5) Construcción basada en las funcionalidades, realizado en iteraciones

2.3.5 Test Driven Development (TDD)

El desarrollo guiado por pruebas es una metodología que se basa en la definición de los ciclos de prueba en primer lugar, es decir, precisa que los requisitos se transformen en guiones de pruebas. Una vez escritas las pruebas, se verifica que fallan (puesto que aún no se ha desarrollado nada), y entonces se implementa el desarrollo necesario para que funcione. De esta forma con la realización de las pruebas se garantiza que la funcionalidad desarrollada cumple con los requisitos.

El ciclo de TDD consta de los siguientes pasos:

- 1) Definir el requisito
- 2) Definir las pruebas para ese requisito
- 3) Verificar que la prueba falla
- 4) Implementar el requisito
- 5) Ejecutar las pruebas (de forma automatizada)
- 6) Refactorizar para eliminar la duplicidad de código
- 7) Actualizar los requisitos

2.3.6 Behavior Driven Development (BDD)

El BDD o desarrollo guiado por comportamiento es un proceso de desarrollo que surge desde el TDD y se puede considerar una extensión del mismo, pues emplea las técnicas y principios de TDD. Su diferencia fundamental es que concreta para la definición de requisitos el uso de un lenguaje de dominio específico que emplea construcciones en lenguaje natural, fácilmente entendibles por los usuarios de negocio. En este lenguaje se expresa un comportamiento y los resultados que se espera obtener.

Estas construcciones en el mundo ágil se conocen frecuentemente como historias de usuario o funcionalidades (*features*) y siguen un patrón predefinido para describir los requisitos y los criterios de aceptación. Un ejemplo común es el siguiente:

- Requisito: Como [rol] quiero [funcionalidad] para que [beneficio]
- Criterios de aceptación: Dado [contexto], cuando [evento], entonces [resultado]

Este lenguaje estructurado permite obtener rápidamente guiones de pruebas ejecutables desde estos requisitos definidos en lenguaje natural, siguiendo entonces el mismo ciclo descrito para el TDD. Las ventajas son evidentes: los criterios de aceptación y las descripciones de la funcionalidad realizadas a nivel de negocio quedan cercanamente relacionados con las pruebas técnicas que lo verifican.

2.3.7 Dynamic Systems Development Method (DSDM)

DSDM es una metodología que se originó en la década de los noventa del siglo pasado, con el objetivo de crear un marco común para la entrega rápida de aplicaciones (RAD).

Se basa en una serie de principios básicos:

- Centrarse en la necesidad y el valor para el negocio
- Involucración del usuario
- Equipos empoderados

- Entregas frecuentes
- Desarrollo iterativo e incremental
- Pruebas integradas
- Colaboración entre las partes

2.3.8 Crystal

Le metodología Crystal es una de las más sencillas, ligeras y adaptables de todas. En realidad, se trata de una familia de metodologías, que incluye entre otras Crystal Clear, Crystal Yellow o Crystal Orange, que varían en ciertos factores como el tamaño del equipo, la criticidad del sistema y las prioridades del proyecto.

Crystal resalta el valor del trabajo en equipo, la comunicación, la simplicidad y la reflexión sobre el proceso, para mejorarlo y adaptarlo. También promueve la entrega rápida, una alta involucración del usuario en el proceso, la flexibilidad y la reducción de la burocracia.

2.3.9 Lean

Lean es una metodología que se tratará con detalle más adelante en este tema. El desarrollo se centra en la entrega de valor al cliente, a través de la llamada cadena de valor (*value stream*), eliminando el desperdicio, o las tareas que no aportan valor al producto final.

2.4 Situación de *Agile* en las organizaciones

En 2017 se publicó el undécimo informe sobre la situación de *agile*, promovido por la empresa VersionOne. Este informe incluye información de miles de participantes encuestados de todas las industrias, tamaños de empresa y geografías.

En él se refleja que el 94% de los encuestados trabajan en organizaciones que usan metodologías ágiles. Sin embargo, aún hay margen de crecimiento, puesto que más de la mitad (60%) de los equipos de estas organizaciones aún no están usando esas prácticas ágiles. Además, el 80% indicó que su organización aún estaba en un nivel bajo de maduración.

Scrum es con diferencia la metodología más seguida (58%). Si se añaden los híbridos de Scrum con Kanban o XP, el porcentaje aumenta a casi 8 de cada 10 proyectos o iniciativas.

Por último, entre los beneficios más reconocidos de *agile* por los propios encuestados, se destacan por este orden:

- La capacidad para gestionar prioridades cambiantes
- La visibilidad del proyecto
- La mejora en la productividad del equipo
- La velocidad de entrega o tiempo de llegada al mercado
- La moral del equipo
- La armonización entre negocio y TI

2.4.1 Situación en la Administración

Las metodologías ágiles se utilizan ya en muchos organismos y proyectos de las Administraciones Públicas, generalmente en entornos cambiantes que requieren una adaptación rápida de los sistemas. Es por tanto una realidad y hay múltiples experiencias de éxito de gestión de proyectos ágiles.

Una de estas experiencias la proporciona el proyecto de desarrollo de la aplicación web SOROLLA2, un servicio común provisto como SaaS por la Intervención General del Estado (IGAE) para facilitar la gestión económica y presupuestaria en los centros gestores del gasto de la administración estatal. SOROLLA2 se utiliza en órganos gestores de ministerios, entidades públicas y otros órganos constitucionales, además de en todas las delegaciones provinciales, contabilizando cerca de 7.000 usuarios.

Las funcionalidades provistas propician un entorno dinámico que requiere una adaptación y flexibilidad para acomodar modificaciones y nuevas solicitudes en tiempos reducidos. Las metodologías ágiles permiten a la IGAE minimizar el tiempo transcurrido entre la definición de la necesidad y la entrega a producción de una versión básica de la solución, que se va evolucionando de forma iterativa.

La metodología ágil escogida para el proyecto de SOROLLA2 es Scrum. Las funcionalidades solicitadas se agrupan en *sprints* de dos semanas, en los que trabajan de forma coordinada varios equipos ágiles. Los despliegues a producción se realizan con frecuencia y para cada despliegue se realizan más de cien casos de pruebas automatizadas de regresión, además de un conjunto de pruebas manuales.

3 Ciclo de vida de un proyecto ágil

Existen diversas metodologías y prácticas ágiles en uso, pero para los propósitos de este tema se explorará un ciclo de vida de acuerdo con el definido en el Agile Project Management framework, que consta de las siguientes cinco fases:

- Concebir (*Envision*): se define el proyecto
- Especular (*Speculate*): se planifican las iteraciones
- Explorar (*Explore*): se desarrollan y se prueban las funcionalidades
- Adaptar (*Adapt*): se revisan los resultados de la iteración
- Cerrar (*Close*): se cierra el proyecto

En este tema se empleará esta nomenclatura para denominar las fases, siendo conscientes de que en los proyectos se podrán encontrar las mismas con nombres diversos. Lo importante es comprender que será necesario realizar una fase de *inicio* del proyecto, seguida de una *planificación, ejecución y revisión* realizadas de forma iterativa, para finalizar con una fase de *cierre* del proyecto. Estas fases tienen matices importantes que las distinguen de las fases equivalentes en un proyecto tradicional, de ahí que empleen una nomenclatura diferenciada.

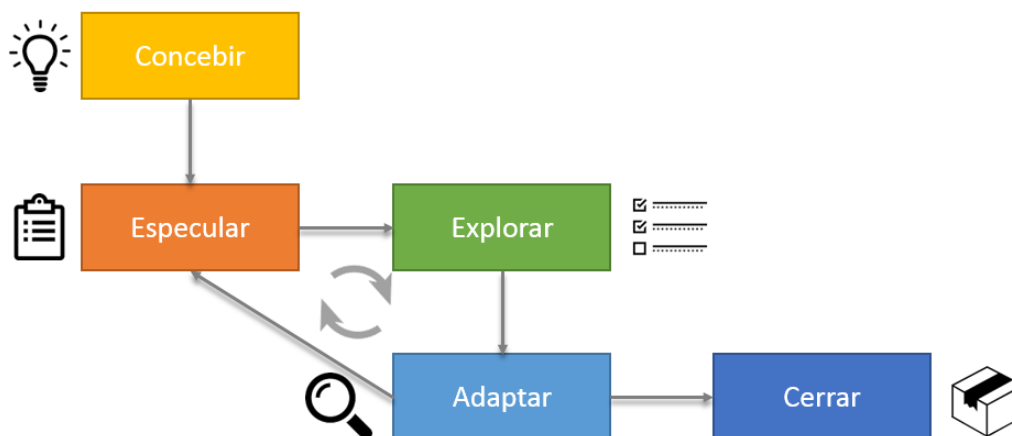


Figura 2 Ciclo de vida ágil

Durante la fase de concepción, en coordinación con el cliente, se definirá el alcance del proyecto, y se determinará la composición de los equipos. Al final de esta etapa se podrá contar con un acta de constitución de proyecto donde se detallará el alcance, objetivos, entregables, participantes y responsabilidades. También se deberán poner en marcha las herramientas de colaboración que se utilizarán durante el proyecto.

A continuación, se pasará de forma iterativa por las siguientes tres fases: especular, explorar y adaptar. Cada ciclo por estas tres fases es lo que denominamos sprint o iteración. Especular supone planificar, creando o revisando un plan de entrega de las distintas funcionalidades (*product features*), estimando el esfuerzo de cada funcionalidad e identificando los riesgos. Al final de la fase de especulación se tendrá una lista de requisitos y una lista de funcionalidades detalladas para desarrollar en esa iteración.

La fase de exploración es donde se realiza el desarrollo y prueba de las funcionalidades. En esta fase se realizan reuniones diarias entre las partes para asegurar el progreso de la iteración, así como frecuentes revisiones entre pares según se va completando el desarrollo de cada funcionalidad.

Una vez que se ha completado el desarrollo se entra en la fase de adaptación, donde se revisan los resultados de la iteración con el cliente y se reflexiona sobre el rendimiento del equipo. En base a estas revisiones y mejoras identificadas, se planifican y ajustan las siguientes iteraciones.

El equipo sigue iterando por estas tres fases hasta completar todos los sprints planificados, y se pasa entonces a la fase de cierre. En esta fase se revisan todos los entregables y se capturan las lecciones aprendidas del proyecto.

A continuación, se recorrerán en detalle estas fases y se analizarán los objetivos y tareas realizadas en cada una de ellas.

3.1 Iniciando el proyecto: Concebir

Esta es la fase que se ha llamado *concebir* o *visionar* en el APM framework.

En esta fase se determina qué se quiere hacer y cómo se va a lograr. Se define el producto que se quiere construir, el alcance y los límites del proyecto, los actores y participantes junto con la distribución de responsabilidades, y se establece la composición del equipo necesario y cómo va a trabajar.

Se genera un documento de constitución del proyecto que defina el alcance y los objetivos generales, e incluido en éste o de forma separada, un diseño a alto nivel que describa la arquitectura del producto a construir (la visión). También ha de identificar las responsabilidades del gestor del proyecto y su nivel de autoridad.

Se ha de detallar el cronograma del proyecto con los hitos fundamentales y elaborar una estimación de los costes totales, tanto de los recursos humanos como materiales. Habitualmente, el proyecto se verá limitado por una serie de factores, como la fecha límite de entrega, el presupuesto, la disponibilidad de los usuarios para participar en el proyecto, la normativa, etc. Todas estas restricciones tendrán que ser consideradas y equilibradas a la hora de realizar el cronograma y estimación de costes. Algunas veces la restricción más limitante es el tiempo, y hay presupuesto suficiente para crear un equipo fuerte y amplio para cumplir los plazos. En otras ocasiones, es el presupuesto, y las características del producto a entregar se verán condicionadas. O también puede ser el equipo, de manera que sólo se disponga de un número limitado de personas y en cambio no exista una restricción temporal severa.

En función del alcance, el tiempo y el presupuesto, se conforma el equipo de proyecto, y se detallan sus normas de trabajo y su ubicación. En *agile* la colaboración cercana entre distintos equipos y entre miembros del mismo equipo es muy importante, y por eso se deben tener equipos con todos los miembros trabajando en la misma ubicación. Sólo en casos excepcionales no será así, y se deberá mitigar con el uso de tecnologías, preferiblemente videoconferencias.

Además, se definen las herramientas de colaboración que se van a emplear, que permitirán compartir información, detallar funcionalidades y requisitos, desarrollar de forma conjunta y realizar el seguimiento del avance. Esto asegurará que todas las partes conocen toda la información y son capaces de colaborar y comentar sobre ella. Hay infinidad de herramientas, grandes y pequeñas, gratuitas o con coste, y cada proyecto deberá determinar sus necesidades de colaboración y las herramientas más apropiadas. Como ejemplo, haciendo uso de las herramientas más utilizadas, se puede construir un marco tecnológico de colaboración empleando *Jira* como herramienta integral de gestión del proyecto, *Confluence* como herramienta para compartir documentación y colaborar sobre la misma y *Slack* para facilitar la comunicación directa por medio de chat o videollamadas.

Una de las tareas más importantes de esta fase es identificar los participantes en el proyecto, tanto de la parte de cliente como de la parte de desarrollo.

3.1.1 El equipo

En las metodologías ágiles se trabaja en un solo equipo, que integra a recursos del cliente y del desarrollo. Cada parte tiene sus roles y responsabilidades claramente marcadas, pero hablar de un solo equipo es una forma de reforzar el sentimiento de colaboración con el que se debe trabajar para alcanzar los objetivos. No obstante, habitualmente (y en este texto) se hace referencia también al *equipo de desarrollo* para identificar a la parte o la unidad dentro de ese único equipo cohesionado que desarrolla el proyecto.

El equipo de desarrollo debe ser pequeño, no más de 10 personas. Si se superase esa cifra, sería conveniente dividirlo en varios equipos con tareas separadas y bien diferenciadas. Se trata de equipos auto-organizados que confían entre ellos y comparten información. La coordinación entre equipos grandes introduciría un elevado riesgo y es una situación que debe evitarse. Para programas o proyectos de gran magnitud, donde deban trabajar varios equipos de manera ágil, se optará por aplicar las metodologías ágiles a gran escala, como se verá más adelante en este tema.

Debe ser multidisciplinar. Deben tener las habilidades necesarias para realizar todas las tareas necesarias para cumplir las funcionalidades requeridas en la iteración, y la colaboración entre todos creará resultados más enriquecedores y de mayor calidad. En lo posible, el equipo no cambiará en todo el proyecto, para fomentar las relaciones personales y para aprovechar la velocidad que se va ganando iteración tras iteración con la experiencia y las mejoras organizativas que el equipo va adoptando.

Además, todo el equipo debe trabajar en la misma localización para poder fomentar los medios de intercambio cara a cara, y se minimizarán otras vías de comunicación menos eficientes y más improductivas como el correo electrónico. La colaboración entre personas es un elemento básico de las metodologías ágiles.

Para ocupar todas las posiciones necesarias en un proyecto ágil, además del equipo de desarrollo se debería contar con los siguientes dos roles: el representante del cliente/negocio y el facilitador.

El representante del cliente o del negocio (conocido como *product owner* en Scrum) es la persona que representa a las partes interesadas en el proyecto y actúa como interlocutor con el equipo de desarrollo. Define los objetivos, crea y mantiene la lista de requisitos y conoce el valor que aporta cada uno de ellos, de forma que puede priorizarlos y distribuirlos en iteraciones. Estará siempre disponible para responder al equipo todas las dudas que puedan surgir en la comprensión de los requisitos y los criterios de aceptación de los mismos.

El facilitador (conocido como *scrum master* en Scrum) lidera al equipo en la aplicación de los valores y procesos ágiles, guiando la colaboración y resolviendo los problemas que impidan el avance del equipo. Coordina las distintas reuniones de inicio, ejecución y cierre de cada iteración. En definitiva, procura que la eficiencia del equipo sea óptima y se encarga de que pueda estar totalmente centrado en su trabajo.

¿Y la figura de gestor de proyecto o *project manager*? De forma fiel con la filosofía ágil, no es necesario, puesto que las responsabilidades de gestión están distribuidas: el equipo de desarrollo se auto-gestiona y decide por sí mismo como se identifican y asignan tareas, el facilitador asegura el seguimiento del proceso y la eliminación de obstáculos, y el representante del cliente identifica y gestiona el alcance y la planificación. El facilitador es el rol que más se identifica con las tareas de gestión del proyecto y con lo que podría denominarse gestor de proyecto, y esta sería la respuesta correcta cuando aplicamos las prácticas ágiles en su esencia.

Sin embargo, hay proyectos que mantienen esta figura de forma separada, especialmente proyectos grandes o en organizaciones complejas. En este caso, su papel será fundamentalmente el de asegurar que todo el equipo está alineado con los objetivos del proyecto, que se tienen en cuenta estos en todas las decisiones fundamentales, y que las distintas partes se coordinan de manera adecuada para lograrlos. Además, debe tener la suficiente autoridad dentro de la organización para asegurar el éxito del proyecto resolviendo los problemas que el facilitador no pueda resolver. Un facilitador tiene autoridad sobre el proceso ágil que se lleva a cabo, pero normalmente no tiene la responsabilidad última sobre el proyecto, sus costes, riesgos, personal, etc. que un gestor de proyecto sí tiene. Si el proyecto fluye de forma natural su papel será secundario, pero si surgen problemas, cambios, o conflictos que por un motivo u otro el equipo no sea capaz de gestionar, su figura aportará tranquilidad para los responsables últimos de la organización.

3.1.2 Duración de las iteraciones

En los proyectos ágiles, la duración de las iteraciones es variable, depende de las necesidades de los grupos de interesados y de las características propias del proyecto.

¿Qué principales ventajas aporta una iteración corta? Principalmente que se entregan resultados con mucha frecuencia, lo cual es positivo en tanto en cuanto permite al cliente estar completamente enganchado al proyecto y estar recibiendo valor de forma continua, y al equipo de proyecto recibir

feedback con esa misma alta frecuencia, reduciendo al mínimo el trabajo en vías erróneas o insatisfactorias y por tanto minimizando el trabajo desperdiciado. Entre sus desventajas destacan la sobrecarga en el tiempo de gestión, puesto que cada iteración lleva aparejada una serie de reuniones previas y posteriores de planificación y análisis de resultados. Además, una iteración corta no es posible en todos los proyectos, puesto que requiere un entorno donde los cambios sean rápidos de implementar. No tiene sentido realizar un sprint corto cuando no se percibe un valor real entre una entrega y la siguiente.

En cambio, una iteración larga permite que el equipo de proyecto esté más centrado en los avances, los cuales puede realizar con más tiempo para sobreponerse a los problemas que pueda encontrar, y precisa menos reuniones de gestión de la iteración. También para el cliente exige menos tiempo de participación en el proyecto. Asimismo, si el equipo de proyecto no tiene tamaño o experiencia suficiente para producir a un ritmo rápido, resulta necesario realizar iteraciones más largas.

Es conveniente que la duración del sprint sea un parámetro fijo. No obstante, al inicio del proyecto, si después de las primeras iteraciones la duración establecida no está funcionando, se debe adaptar. Eso sí, una vez encontrada la duración óptima, conviene no cambiarla para que todas las partes se acostumbren a esa frecuencia y adapten sus ritmos de trabajo a la misma. Además, no conviene que las fechas de entrega sean un parámetro en permanente discusión, al contrario, es preferible que sea algo previsible que todos conozcan de antemano.

En la tabla siguiente se intentan reflejar las características del proyecto, del cliente o del equipo de proyecto que se verán mejor acomodadas en iteraciones más cortas o más largas. Estas claves ayudarán a encontrar de partida la duración de la iteración más conveniente para un proyecto.

	Cliente	Equipo	Proyecto
Iteraciones cortas	- Acostumbrado a trabajar de forma ágil - Muy involucrado - Solicita entregas frecuentes - Exigente y perfeccionista	- Experto en la tecnología usada - Bien formado - Acostumbrado a trabajar de forma ágil	- Sistema existente o paquete configurable, en el que se puedan realizar modificaciones rápidas con resultados relevantes - Entorno variable y dinámico - Necesidad de obtener resultados rápidos, aun cuando proporcionen sólo la funcionalidad básica - Requisitos difusos, con riesgo de no saber o interpretar bien lo que quiere el cliente
Iteraciones largas	- Inexperto o poco acostumbrado a trabajar de forma ágil - Poco tiempo disponible en el día a día - No presiona por tener entregas frecuentes - Flexible con el resultado, mientras se cumpla el objetivo final	- Poco experimentado en la tecnología usada - Recursos escasos - Poco acostumbrado a trabajar de forma ágil	- Requisitos claros y precisos, baja incertidumbre y por tanto bajo riesgo de retrabajar en cosas ya realizadas porque no satisfagan al cliente - Necesidad de obtener resultados completos - Sistema complejo, donde se requiera mucho trabajo para realizar avances o haya que buscar soluciones creativas - Proyecto largo, puesto que en un proyecto corto con iteraciones largas las oportunidades de recoger feedback serían muy escasas

Figura 3 Claves para determinar la duración óptima de la iteración.

En caso de desconocer información o tener factores enfrentados que dificulten la elección, como término medio se puede determinar la duración en 3 o 4 semanas. Es una duración común en proyectos ágiles y que se adapta a situaciones diversas.

Si se dan todos los factores necesarios se podría reducir la duración a 2 semanas, o incluso a 1 semana. Debería ser un proyecto con un sistema fácilmente modificable, probablemente por medio de configuraciones rápidas, y con un equipo y un cliente ya acostumbrados a trabajar de forma ágil.

En el extremo superior, más de 4 semanas no es frecuente (hay metodologías como Scrum que de hecho limitan la duración máxima de los sprints a 4 semanas) y no resulta recomendable apostar por iteraciones más largas a menos que se tengan bien claras la justificación y la descripción del flujo de trabajo que precisa de esos tiempos más largos. En este caso, se podría extender hasta las 8 o 10 semanas, normalmente en proyectos grandes con equipos menos nutridos o más inexpertos en metodologías ágiles y con sistemas que requieran un tiempo exploratorio y creativo, o un trabajo intensivo para poder ver avances significativos.

Por último, hay que entender que como norma general cuanto más corta sea la iteración, más se intensifican los beneficios de *agile*. Si se trata de un equipo experto con todos los condicionantes favorables para trabajar de forma eficaz en iteraciones muy cortas, no sería razonable extender la duración de forma injustificada.

3.1.3 Herramientas de gestión de proyectos ágiles

Además de Atlassian Jira, citada anteriormente, existen otras herramientas muy utilizadas en gestión de proyectos ágiles como son Microsoft Excel (un clásico en todas las áreas por su sencillez), VersionOne, Microsoft Team Foundation Server (TFS) o CA Agile Central.

Realizar un análisis de todas las herramientas para determinar cuál es la más adecuada para cada proyecto y organización es muy laborioso, sin embargo, se pueden aportar algunas pautas. En primer lugar, dependerá de si la organización ya utiliza prácticas ágiles, lo cual es muy habitual hoy en día, y en ese caso lo normal será usar la que ya esté siendo usada en la organización, por comodidad, conocimientos y gestión de licencias.

Si en el equipo hay muchos desarrolladores, es probable que la decisión se incline hacia Jira, puesto que hay bastantes opciones de que ya la estén usando. Jira es la herramienta más utilizada y tiene una funcionalidad muy completa, además de integrarse fácilmente con herramientas como Confluence, que mejora la gestión de la documentación.

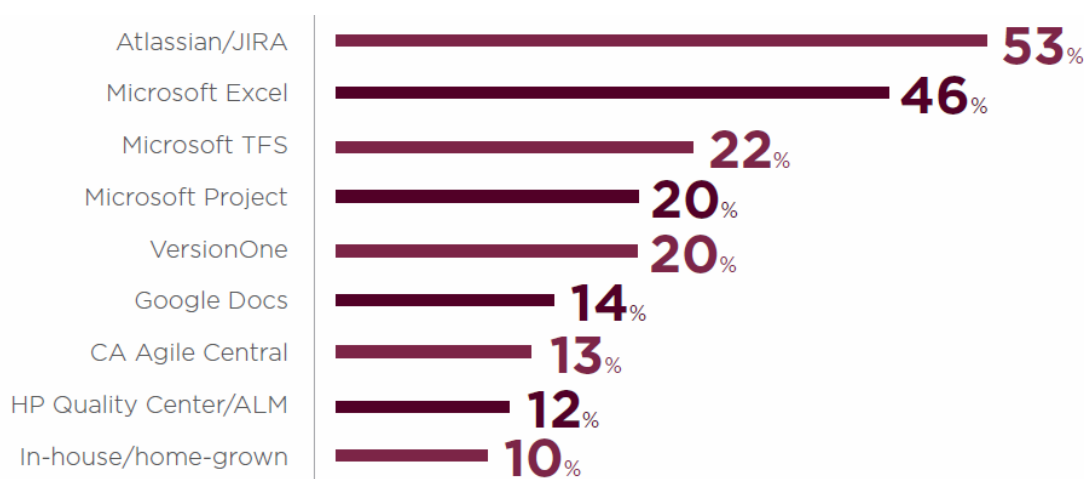


Figura 4 Herramientas ágiles más usadas. Fuente: 11º Informe anual del estado de agile (VersionOne)

Si la organización apuesta por herramientas de Microsoft, entonces TFS le resultará familiar a muchos de los equipos. CA Agile Central es una herramienta menos popular sobre todo entre los usuarios de negocio por ser más compleja de usar y de implantar, siendo por tanto adecuada en entornos con

personal fundamentalmente técnico. VersionOne por su parte es adecuada si se está comenzando con la transición hacia las prácticas ágiles. Y por último Excel, o equivalentes como Google Sheets, es una herramienta que todo el mundo conoce, no precisa ningún tiempo de aprendizaje, y puede ser un punto de partida válido si las necesidades de gestión son sencillas.

3.2 Planificando el trabajo: Especular

Una vez está el proyecto definido y documentado en su alcance y objetivos, las herramientas de colaboración están operativas y el equipo preparado, el proyecto puede avanzar a planificar la primera iteración.

Esta es la fase que se ha llamado *especular* en el APM framework. Que se escoja ese nombre en lugar del de planificación ya deja claro el matiz que lo diferencia de la planificación tradicional. Especular supone aceptar que el plan no marca el proyecto, que las desviaciones del plan por tanto no serán vistas de forma negativa, sino que de hecho se espera que las necesidades cambien a lo largo del proyecto. Por tanto, lo que se puede hacer es una especulación, que marque la dirección y el objetivo, y un plan que posteriormente se irá revisando. La especulación resulta una ayuda para marcar el rumbo cuando se está iterando, pero no supone un corsé para el proyecto.

El objetivo de esta etapa es el de identificar las funcionalidades que formarán parte de la iteración en curso. Este será un trabajo coordinado entre el negocio, que conoce los requisitos y su valor, y el equipo de desarrollo, que aporta el conocimiento de su complejidad y el esfuerzo que requiere su implementación. Hay que tener en cuenta las funcionalidades de la lista creada en la fase de concepción, y, si no es la primera iteración, también las funcionalidades nuevas que hayan podido surgir por el camino, así como las que no se hayan podido completar en la iteración anterior.

En este punto las funcionalidades deben estar disgregadas en pequeños fragmentos individuales, que se puedan gestionar y entregar de forma diferenciada, que estén centrados en una necesidad de negocio y que aporten valor al cliente. Se expresan normalmente en forma de acción – resultado. En Scrum se conocen como historias de usuario. Las historias de usuario son una manera de declarar una funcionalidad requerida con descripciones directas desde el punto de vista de un usuario. Algunos ejemplos podrían ser:

- Como gestor de facturación, al imprimir una factura, quiero ver las direcciones de envío y de facturación en el encabezado.
- Como cliente habitual, quiero ver productos relacionados al que estoy comprando para saber si me pueden interesar.

Las funcionalidades se agrupan lógicamente por temas para facilitar su organización (en el ejemplo anterior, podría haber un tema que fuera *facturas*).

En la primera iteración, el equipo debe proporcionar estimaciones de esfuerzo para todas las funcionalidades de la lista. Las estimaciones se pueden hacer de forma relativa, en comparación con el resto de funcionalidades, asignando puntos a cada una de ellas. Un elemento que se estime en 8 puntos requerirá cuatro veces más esfuerzo que uno estimado en 2 puntos, pero sin necesidad de especificar si un punto corresponde a una hora o diez. Posteriormente se traducirá esto en horas de trabajo, pero la conversión puede ajustarse según se mejoran las dinámicas de trabajo o cuando se analizan los resultados de una iteración. En el primer sprint, quizá se establezca que puedan acomodarse 100 puntos de esfuerzo en una iteración. En cambio, según se avanza y se ve el rendimiento real de iteraciones anteriores, quizá se vea que se pueden realizar 120, o al revés, sólo 80. También se estiman con frecuencia en tamaños, que van desde XS a XL. Realizar la estimación de forma relativa en lugar de horas de trabajo, ayudará a realizar estas adaptaciones de ritmo y planificación de la iteración.

En la primera iteración se creará una hoja de ruta y un plan de entrega de funcionalidades por iteraciones, que definirán de qué forma se agrupan las funcionalidades y cuándo se implementarán y se entregarán. Por ello, la primera fase de especulación será más larga. En el resto de iteraciones, debería ser una fase corta (uno o dos días para sprints de 2-3 semanas).

3.2.1 Registro de requisitos

En las metodologías ágiles el registro de requisitos debe ser flexible y fácil de mantener, porque el cambio y la adaptación del alcance forman parte de su esencia, y ello comporta poder modificar, añadir y eliminar requisitos, así como cambiar su prioridad.

El representante del cliente (o *product owner*) es el propietario de los requisitos. Es su responsabilidad que se mantengan actualizados y con un nivel de detalle suficiente para acometer su desarrollo, especialmente los que se vayan a implementar en la siguiente iteración. Normalmente trabaja por delante del equipo de desarrollo, junto con los usuarios de negocio, para ir detallando y aclarando los requisitos, o añadiendo nuevos si es necesario, y que así estén listos para cuando empiece el sprint que debe desarrollarlos. También puede identificar requisitos que ya no son necesarios y deban eliminarse.

Los requisitos se documentan en forma de funcionalidades e historias de usuario como se ha indicado anteriormente. Éstas se añaden a una lista de elementos pendientes, o *product backlog*. Todo el trabajo pendiente debe estar presente en el *backlog*, incluidas las tareas técnicas como puede ser la actualización de versión de un servidor, o tareas de investigación y exploración de distintas opciones. El *backlog* normalmente se gestiona en una aplicación que ayuda a detallarlo, ordenarlo y realizar un seguimiento del mismo, y que favorece que toda la información sobre los requisitos esté a disposición de todas las partes. Junto a un nombre corto y una descripción detallada del requisito, se debe incluir también unos criterios de aceptación. En estos criterios se ha de definir la forma de validar que el requisito ha sido implementado con éxito. Ejemplos de criterios de aceptación bien definidos pueden ser:

- Historia de usuario: como gestor de facturación, al imprimir una factura, quiero ver las direcciones de envío y de facturación en el encabezado.
 - o Criterio de aceptación → La dirección siempre estará expresada con los siguientes campos: calle, número de calle, piso, código postal, ciudad y país.
- Historia de usuario: como cliente habitual, quiero ver productos relacionados al que estoy comprando para saber si me pueden interesar.
 - o Criterio de aceptación → Deberá mostrarse el nombre del producto, dos líneas de su descripción y la fotografía. Si el usuario hace clic sobre el producto, se abrirá su ventana descriptiva. En caso de que hubiera varios productos relacionados, estos estarán ordenados según la clasificación de prioridad asociado a cada uno.

Todos los requisitos deben ser priorizados por el cliente en función del valor que aportan a su negocio. Esta priorización debe ser más intensa en los elementos que están en la parte alta de la lista, puesto que los de la parte baja no se desarrollarán próximamente y habrá tiempo de afinar su priorización.

La prioridad para el negocio, unida al factor de complejidad de realización estimada por el equipo de desarrollo, definirá un retorno de inversión mayor o menor. Generalmente, los requisitos con mayor retorno de inversión (es decir, que con poco esfuerzo aportan un gran valor) serán los que estarán al principio de la lista y los que por tanto se implementarán con mayor prioridad, siempre que no haya otras necesidades de negocio que fuercen a realizar otros requisitos más críticos independientemente de su complejidad. Otro de los factores que influyen en la priorización es el riesgo que se identifica en la realización de esa funcionalidad. Elementos con bajo valor y alto riesgo serán evitados por el cliente.

Hay que tener en mente que no todos los requisitos del *backlog* llegarán a implementarse en el proyecto en curso debido a limitaciones de tiempo y presupuesto, y por tanto la priorización es una labor fundamental.

3.2.2 Distribución de funcionalidades

Una de las cuestiones que se deber resolver en la planificación de la primera iteración, es acerca de cuál es la manera más óptima de distribuir las funcionalidades a entregar en cada iteración.

En primer lugar, como se ha indicado antes, la prioridad establecida por el retorno de inversión de cada funcionalidad es un elemento clave que influirá de forma decisiva en la definición del orden de implementación. Pero hay más elementos que se pueden considerar.

Así, por ejemplo, lo más conveniente es que se creen grupos de funcionalidad y que cada uno de ellos se entregue de la manera más cohesiva que sea posible. De esta forma se ahorrará el esfuerzo de retomar una y otra vez la misma área. Por tanto, si hay varias funcionalidades prioritarias que se añaden a la iteración, puede resultar conveniente añadir también otras menos prioritarias si conforman un mismo grupo.

También puede haber restricciones técnicas que, independientemente de la prioridad para el negocio, fuercen a que unas funcionalidades se realicen antes que otras puesto que suponen un pre-requisito para poder desarrollarlas.

Por otro lado, debería plantearse cómo reducir el riesgo que supone un equipo nuevo, con mecanismos de trabajo aún por engrasar, y conjugarlo con la prioridad y el valor establecido por el cliente para cada funcionalidad. Una posible estrategia es la de dedicar el primer sprint a características menos complejas e intrincadas, para dar tiempo a los componentes del equipo a adaptarse a los demás y a la forma ágil de trabajo. En cambio, si se cuenta con un equipo experimentado en *agile* y que además ya ha trabajado de forma conjunta en un proyecto anterior, una estrategia adecuada sería la de programar para las primeras iteraciones las funcionalidades más complejas y valoradas, para evitar sorpresas en las iteraciones finales.

También es habitual que otras necesidades del cliente restrinjan la libertad para distribuir las funcionalidades. En función del impacto y de los ciclos del negocio, puede haber un requerimiento para entregar las funcionalidades en un momento determinado: por ejemplo, posponiendo la entrega de una característica de alto impacto por la cercanía de vacaciones o para no interferir en la campaña de navidad.

Una vez que las funcionalidades han sido revisadas por el cliente y priorizadas con la información aportada por el equipo de desarrollo, todo está listo para comenzar a ejecutar el sprint. Las funcionalidades por desarrollar dentro de la siguiente iteración conformarán el *backlog* individual de esa iteración (conocido como *sprint backlog* en Scrum).

3.3 Construyendo el producto: Explorar

Esta es la fase que se ha llamado *explorar* en el APM framework. Es donde se va creando el producto, desarrollando y probando las funcionalidades.

El punto de partida es el *backlog* de la iteración, que comprende las funcionalidades que se han de implementar durante esas semanas. En el *backlog* de la iteración, el equipo de desarrollo descompondrá cada elemento en las tareas que son necesarias para su implementación. Esta división en tareas además de guiar el desarrollo también ayuda a estimar el esfuerzo necesario.

Entre esas tareas que se han de llevar a cabo para cada funcionalidad estarán las de diseñar la lógica o la interfaz, desarrollar el código y realizar las pruebas unitarias y de regresión. También es común realizar algún tipo de documentación funcional y técnica con un enfoque práctico que se va completando en paralelo a las iteraciones y donde se reflejan las funcionalidades implementadas. Esto permite que al finalizar el proyecto quede un diseño completo y actualizado, más allá de los diseños de alto nivel que se hicieron a priori y de las historias de usuario, y que será de ayuda en sucesivos proyectos o en la explotación del sistema. En este caso, una de las tareas a introducir sería también la de completar la documentación con la nueva funcionalidad.

Las funcionalidades por desarrollar tradicionalmente se apuntaban en tarjetas que se pegaban en una pizarra, y se iban moviendo según se avanzase de estado hasta cerrarlas. Hoy día, esto se simula con herramientas de gestión como *Jira*, pero el mecanismo es equivalente. Los miembros del equipo de desarrollo se asignan una funcionalidad, que desarrollan y pasan al siguiente estado. Así, todos los miembros del equipo pueden conocer el grado de avance de la iteración de un vistazo.

El papel de gestor de proyecto es marcadamente distinto a la de un proyecto tradicional. No se asignan tareas, el equipo se auto-organiza y se auto-gobierna y la labor del gestor se reduce a asesorar y eliminar impedimentos y bloqueos para asegurar que el equipo tiene todo lo que necesita para avanzar

y que de esta forma se pueda concentrar en su trabajo. Por ese motivo se le asigna el papel de facilitador.

La colaboración constructiva es primordial y por ello los equipos son más efectivos si se encuentran en la misma ubicación. Durante las iteraciones, los desarrolladores validan su trabajo con otros miembros del equipo, habitualmente en parejas, para verificar tanto su enfoque, como, una vez completado, el resultado.

Durante la iteración se realizan reuniones diarias donde están presentes todos los miembros del equipo, incluidos el representante del cliente y el facilitador, y donde cada miembro del equipo de desarrollo brevemente presenta el avance de las tareas: qué se hizo ayer y qué se planea hacer hoy. Además, indicarán si precisan ayuda o tienen algún impedimento que les bloquee el avance. Las reuniones diarias no deberían exceder los 10 o 15 minutos, por lo que no es lugar para resolver los problemas que se identifiquen. En Scrum se denominan *stand-up meetings* puesto que transcurren con todos de pie, lo que incentiva que la reunión sea corta y se vaya al grano.

El facilitador tomará los impedimentos identificados y liderará su resolución fuera de la reunión, junto con las personas imprescindibles para solventarlo. Es el gestor de la eficiencia del equipo, que debe procurar asegurar y mejorar la productividad del mismo, y las reuniones diarias le aportan la visión que necesita para hacer su trabajo. Además, le permite mantener el control del proyecto y seguir más de cerca el progreso de la iteración frente al plan.

La fase de exploración finaliza cuando se consume el tiempo planificado para esa iteración. Si no se han completado todas las funcionalidades que se programaron, se añadirán a la planificación de los siguientes sprints. Y si se completan todas las funcionalidades previstas antes del final de la iteración, se continúa con las siguientes de la lista de funcionalidades pendientes, aquellas que inicialmente se deseaba acometer pero que habían quedado fuera de la iteración por falta de capacidad y que estarán asignadas a la siguiente iteración.

3.3.1 Control del avance

Como se ha visto antes, la lista de *backlog* del proyecto es la lista de requisitos priorizada y refleja el alcance del proyecto. Esta lista es dinámica, y es normal que se vayan añadiendo y modificando elementos durante el proyecto. Una vez comienza una iteración se asigna un grupo de funcionalidades para ser desarrollados durante la misma, creándose el *backlog* particular de esa iteración, que, a diferencia de la lista general, es una lista estática y que no se debe modificar durante la iteración.

El *backlog* del proyecto (también llamado *backlog* del producto) debe ir disminuyendo según se avanza en el tiempo y se van desarrollando funcionalidades. Se monitoriza en forma de gráfica, donde se confrontan las funcionalidades pendientes con el tiempo del proyecto, y se compara con una función lineal ideal donde la velocidad de resolución fuera constante. De esta manera de un vistazo se puede ver si el proyecto está en tiempo o no. Es lo que se denomina gráficos de trabajo pendiente (*burndown charts*), e igualmente se realizan para monitorizar el avance a nivel de cada iteración.

La velocidad del equipo se entiende como la cantidad de trabajo (medida normalmente en puntos) que el equipo realiza en una iteración. Se obtiene una vez se han desarrollado los primeros sprints, y se monitoriza para conocer si el equipo la mantiene o incluso la aumenta, lo cual sería lo más saludable, o bien si disminuye su ritmo a lo largo del tiempo, lo cual puede ser señal de algún problema que habrá que identificar y resolver.

Es importante medir las funcionalidades según su nivel de esfuerzo (ya sea en puntos o si se desea en horas) y no en número para poder representar un progreso realista, dado que lógicamente hay funcionalidades más sencillas que otras. En el siguiente gráfico se muestra como podría ser un gráfico de trabajo pendiente para un proyecto de diez iteraciones que se encuentra en la octava ligeramente por delante de lo planificado.

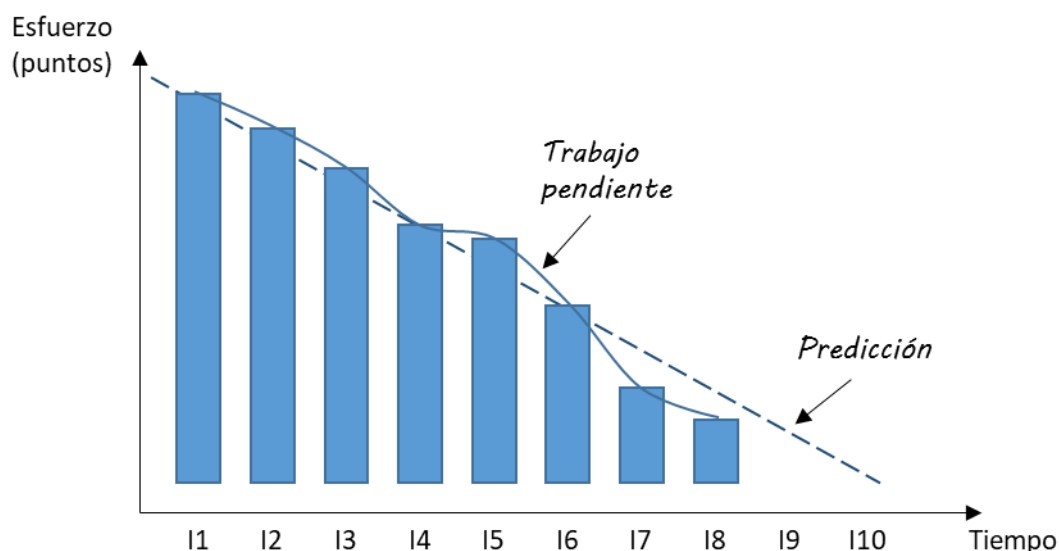


Figura 5 Gráfico de trabajo pendiente (*burndown chart*) para un proyecto de diez iteraciones.

El mismo gráfico a nivel de una iteración sería similar, cambiando la escala temporal para mostrar días en lugar de iteraciones, y reduciendo el esfuerzo total al alcance que se haya incluido en esa iteración.

Además del *burndown chart*, también se emplea otro tipo de gráfico que muestra la evolución del valor entregado al cliente. Se denomina *burn-up chart*, y aunque es menos conocido, también es de gran utilidad para mostrar el alcance y el trabajo completado. En el gráfico se muestra la evolución de dos líneas: por un lado, el alcance (que también evoluciona) y, por otro lado, el trabajo completado. Ambos se suelen medir en puntos o la unidad de esfuerzo que se utilice en el proyecto. Cuando ambas líneas se encuentran, quiere decir que el *backlog* se habrá implementado por completo. Se utiliza para sacar una proyección de cuando terminaría el proyecto si se deseara completar toda la funcionalidad, considerando tanto el ritmo de trabajo como el ritmo de variación del alcance (y esta es la gran ventaja frente al *burndown chart*).

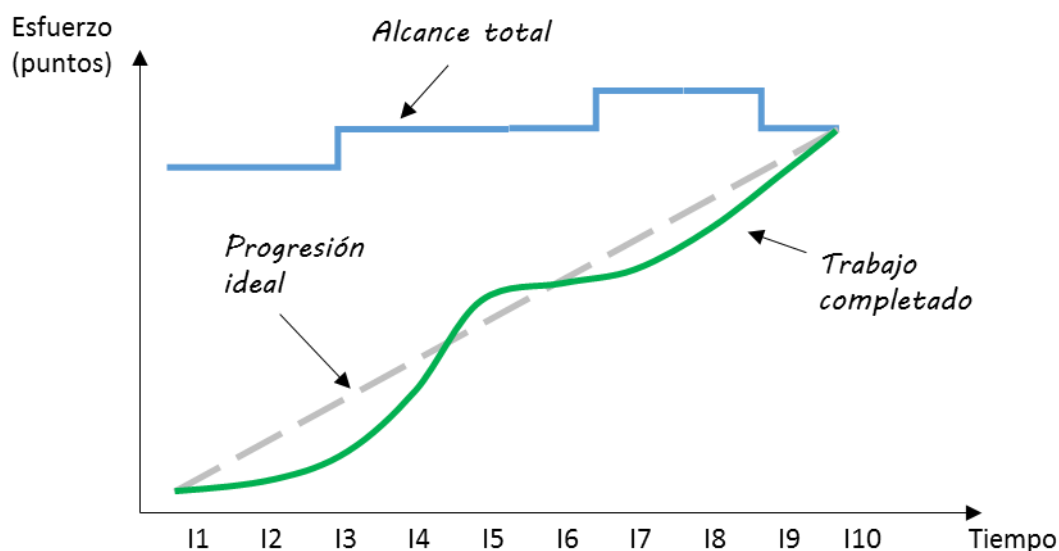


Figura 6 Gráfico de trabajo realizado (*burn-up chart*) para un proyecto de diez iteraciones.

La información proporcionada por estos gráficos se usará para realizar ajustes en el proyecto. Si se está por delante de lo programado, se podrán añadir nuevas funcionalidades, o terminar el proyecto

antes de lo planificado. Si por el contrario se va por detrás de lo programado, se debe extender el proyecto o replanificar las funcionalidades que se van a desarrollar.

3.4 Revisando el progreso: Adaptar

Esta es la fase que se ha llamado *adaptar* en el APM framework. Ocurre al final de cada iteración, una vez acabado el desarrollo de las funcionalidades. Materializa una de las grandes características de *agile*: la oportunidad de ver resultados rápidamente para recoger feedback y poder adaptarse a requisitos cambiantes.

Entre sus objetivos está revisar con todo el equipo lo que se ha desarrollado, así como reflexionar sobre posibles puntos de mejora en la forma en que se está trabajando y acordar los cambios que ayudarán al equipo a rendir mejor.

En esta fase se muestra al cliente el estado del producto que se está creando, por medio de sesiones de demo donde se navega por el sistema y se realizan en tiempo real las funcionalidades incorporadas tal y como las realizaría un usuario. En estas sesiones participa todo el equipo junto con el sponsor y todas las partes interesadas. Constituyen un momento clave para recoger el feedback del cliente puesto que viendo el sistema en funcionamiento apreciará facetas de la solución que es difícil tener presentes sobre el papel, como la manera en que se fluye por el sistema, la apariencia de las pantallas, el número de acciones necesarias para conseguir los objetivos, etc. Esto puede dar lugar a rectificaciones sobre cosas que ya estaban construidas, para ajustarse mejor a las necesidades del negocio. Asimismo, ver la aplicación en funcionamiento y las posibilidades que ésta ofrece, inspira también para reformular o crear nuevos requisitos que se desearía tener. Por supuesto también pueden surgir requisitos a partir de nuevas necesidades de negocio que no estaban presentes al inicio del proyecto (por ejemplo, el competidor puede lanzar un sistema similar pero con una llamativa funcionalidad que desde ese momento también se desee ofrecer). Todo ello originará la adición de nuevas funcionalidades a la lista de *backlog*.

Ésta es la esencia de las propuestas ágiles y no debe encrespar al gestor. Los proyectos ágiles, como los tradicionales, tienen limitaciones de tiempo y presupuesto. Todas las partes involucradas en el proyecto asumen que habrá funcionalidades que tendrán que ser pospuestas a futuros proyectos por este motivo. Si se incluyen requisitos no previstos, probablemente durante el ejercicio de priorización se quedarán otros relegados al final de la lista que no podrán ser acometidos.

Además de estas sesiones de demostración, a continuación se organizan reuniones retrospectivas donde participa únicamente el equipo (en sentido amplio, incluyendo el representante del negocio y el facilitador), puesto que son por y para la mejora del funcionamiento del equipo. En estas sesiones se analiza el resultado de la iteración frente a lo previsto y se discuten problemas u oportunidades de mejora para ser incorporadas a la siguiente iteración y mejorar la eficiencia del equipo. Éstas pueden ser de todo tipo, desde cambiar el horario de las reuniones diarias o la manera en que se describen las funcionalidades, a otras de mayor calado, como cambiar el equipo, la duración de los sprints o suprimir algún paso del proceso.

También, como gestor, es un momento para motivar al equipo y reconocer los esfuerzos realizados durante la iteración.

La fase de revisión no debería llevar más de 1 o 2 días en proyectos ágiles de duración intermedia. Una vez completada, si no es el final del proyecto, se procede a realizar nuevamente la fase de especulación de la siguiente iteración. Si se trata de la última iteración, en ese caso se procede a cerrar el proyecto.

3.4.1 Solución de problemas

Siguiendo la filosofía ágil, la manera adecuada de buscar las soluciones que beneficiarán el rendimiento del equipo parte de la participación de todos, permitiendo que todos los componentes del equipo aporten ideas, se pronuncien y valoren la mejor opción. Una solución adoptada después de escuchar a todos tiene muchas más probabilidades de éxito.

Entre los problemas que pueden surgir en un proyecto ágil se pueden destacar:

- Falta de disponibilidad de los usuarios de negocio: El representante del cliente debe proporcionar respuestas rápidas a los miembros del equipo de desarrollo. Si él no puede proporcionarlas y los usuarios de negocio están muy ocupados en su día a día como para aclarar los puntos conflictivos, se originarán retrasos en el desarrollo de las funcionalidades. El representante del negocio debe trabajar por delante del equipo de desarrollo, para garantizar que tiene las prioridades de negocio asimiladas y puede proporcionar respuestas rápidas y fiables en los momentos en que la disponibilidad del usuario de negocio sea limitada.
- Falta de entendimiento entre el equipo de desarrollo y el cliente: Bien porque el cliente no entienda lo que se está tratando de desarrollar o bien porque el equipo no comprenda los requisitos, esto originará con más frecuencia de la deseada que haya funcionalidades erróneamente desarrolladas y un trabajo malgastado que deba rehacerse. El representante del negocio debe asegurar que las funcionalidades están correctamente descritas y detalladas, y dispuesto para explicar y resolver las dudas que puedan surgir al equipo de desarrollo. También se podrán programar sesiones adicionales con el cliente para que comprenda las tecnologías utilizadas.
- Iteraciones no completadas: Las funcionalidades programadas para cada iteración no se están completando, pasándose por tanto a la siguiente iteración. Esto produce un efecto en cadena, quedándose en cada iteración más funcionalidades sin hacer hasta que el proyecto acumula un retraso que compromete su éxito. Se debe investigar la raíz del problema y poner solución lo antes posible. Puede ser una planificación demasiado optimista, un equipo con poca experiencia en los procesos ágiles o en las tecnologías empleadas, falta de motivación o algún conflicto interno, etc.
- Se invierte demasiado tiempo en reuniones y se reduce el tiempo de desarrollo: La estricta gestión del tiempo es un elemento fundamental en las metodologías ágiles. Puesto que hay muchas oportunidades de colaborar y mantener sesiones conjuntas, éstas deben ser controladas para que no consuman el tiempo del equipo. Se debe identificar a un responsable de mantener el control del tiempo, que puede ser el facilitador o asignarse de forma rotatoria, y que distribuya los elementos a tratar y no permita que se prolonguen las discusiones.

3.5 Finalizando el proyecto: Cerrar

El cierre del proyecto ocurre cuando se realizan todas las funcionalidades en el alcance, o bien cuando se termina el tiempo o el presupuesto adjudicado.

La fase de cierre incluye tareas administrativas, como recolocar a los miembros del equipo en otros proyectos, realizar las tareas de facturación con empresas proveedoras o las propias de la revisión y cierre del presupuesto adjudicado al proyecto.

También es momento de comunicar. Al final del proyecto se ha de transmitir a todas las partes interesadas de la organización los resultados obtenidos, desde el punto de vista de las necesidades de negocio cubiertas y el valor aportado.

Además, si el proyecto ha terminado por falta de tiempo o de presupuesto y se han quedado funcionalidades sin desarrollar, se trabajará con el cliente para determinar la importancia de las mismas y, en su caso, proponer un nuevo proyecto que continúe el trabajo realizado (para lo cual será necesario captar nuevos fondos).

Se documentarán las lecciones aprendidas, para poder ser aprovechadas en los siguientes proyectos.

Y muy importante, como en todo proyecto habrá que realizar un seguimiento y evaluación del cumplimiento de los objetivos durante los meses siguientes, definiendo y monitorizando los indicadores que sean relevantes.

4 Agile a gran escala

Las metodologías ágiles basan su efectividad en la comunicación cara a cara y por tanto están más adecuadas a equipos pequeños. Además, los equipos distribuidos representan todo un desafío, y es aconsejable que todos se encuentren en la misma ubicación.

Para proyectos o programas grandes, en organizaciones complejas, donde se hace necesaria la participación de muchos equipos que se desea que trabajen de forma ágil, surgen las metodologías que aplican *agile* a gran escala. Entre ellas, las más utilizadas con diferencia son SAFe (Scaled Agile Framework) y Scrum de Scrums. SAFe utiliza principios de Scrum, pero también de XP, Kanban y Lean, mientras que Scrum de Scrums escala la metodología Scrum de manera más pura. No obstante, ambas metodologías comparten su filosofía y sus principios generales, que se explicarán en este apartado.

El objetivo es sincronizar el entendimiento, la colaboración y la entrega para un elevado número de equipos ágiles.

En cada equipo se mantienen los mismos principios y organización ya referidos en este tema:

- Equipos empoderados, auto-organizados y auto-gestionados, multidisciplinares
- Realizando entregas cada poco tiempo, probadas y funcionando, y que aportan valor al cliente
- Entregando valor de forma incremental por medio de historias de usuario

Estos mismos principios se escalan, para obtener:

- Un equipo de equipos ágiles auto-organizado y auto-gestionado
- Que entrega incrementos de funcionalidad cada poco tiempo
- Que opera con una visión y una arquitectura común
- Con una duración de las iteraciones común
- Colaborando cara a cara
- Entregando valor de forma incremental por medio de funcionalidades de mayor escala

4.1 Coordinación

Las prácticas de *Agile* escalado permiten el trabajo a nivel de programa, que puede consistir en múltiples proyectos trabajando de forma coordinada. Por ejemplo, se puede pensar en una gran iniciativa que consista en crear un nuevo y complejo portal para los clientes de una entidad financiera. En este marco, habrá equipos que trabajen en la autenticación del usuario, otros en la forma de mostrar la información de sus cuentas, otros en permitir pagos, etc. Estos equipos tendrán objetivos comunes, y por ello deberán trabajar de manera coordinada. En estas metodologías, se recomienda dividir los equipos por área de funcionalidad y no por capa tecnológica como en proyectos tradicionales (lo que sería por ejemplo un equipo dedicado a la interfaz de usuario, otro a la lógica de negocio, otro a las integraciones con otros sistemas, base de datos, etc.). Dividiendo por área funcional y no por capa técnica, se maximiza la velocidad y la calidad y se minimizan las dependencias y las entregas entre equipos.

Cuando se habla de *agile* en escala, se trata de una organización típicamente de unos 5 a 12 equipos (por tanto, de 50 a 120 personas) que planifica, se compromete y ejecuta al unísono. Si se requiriesen más equipos, sería conveniente separar el programa en distintos temas y añadir una capa de coordinación por encima entre los distintos temas.

Para sincronizar a los equipos se definen los *incrementos de programa* o PI por sus siglas en inglés, que son un período de tiempo predefinido donde las iteraciones de cada equipo se deben acomodar. Estos incrementos a nivel de programa suelen ser de unas 10 semanas, y en cualquier caso debe ser un tiempo fijo y conocido con antelación por todos los equipos. Tanto las iteraciones como los incrementos de programa están sincronizados en todos los equipos. Además, hay una misión común, monitorizada por medio del *backlog* del programa.

Es importante destacar que un PI no supone un paso a producción. La cadencia del desarrollo impone estos períodos fijos, pero naturalmente las necesidades de la organización determinarán cuándo se requiere realizar un pase a producción. Éste será bajo demanda en cualquier momento, no tiene por qué coincidir con un PI, y conllevará una serie de actividades encaminadas a poner los incrementos desarrollados en producción.

La siguiente figura representa la cadencia y sincronización entre varios equipos, con un PI cada cuatro iteraciones.

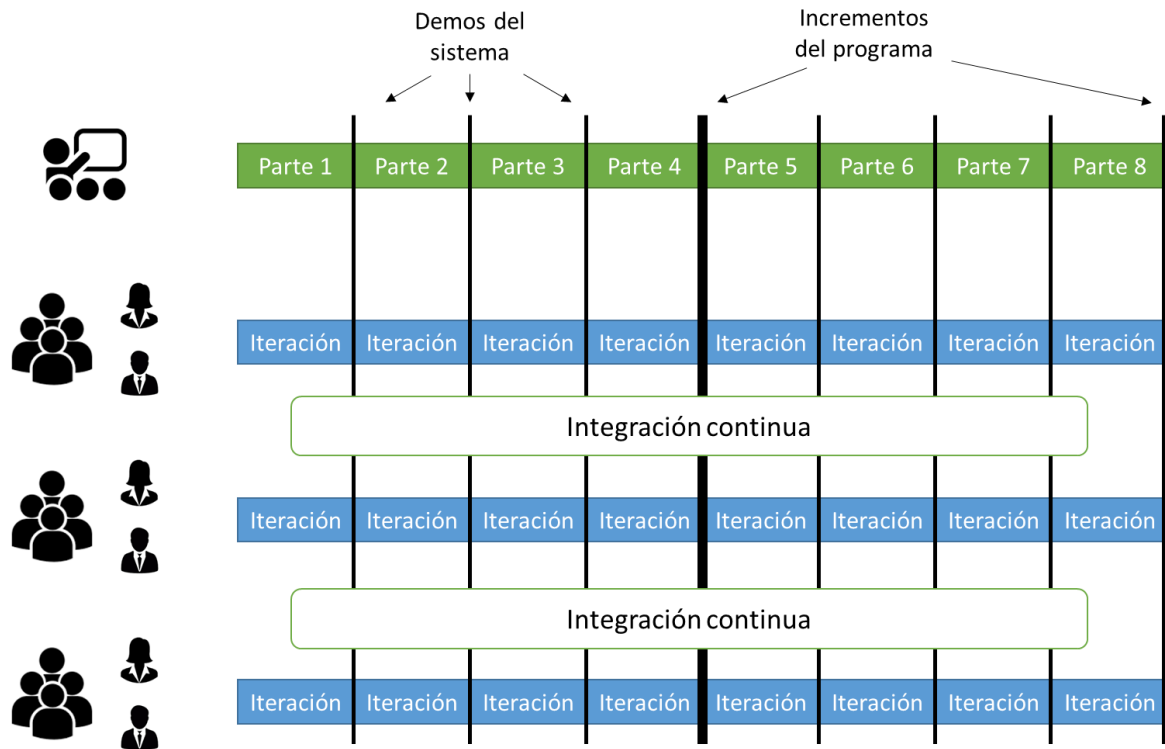


Figura 7 Sincronización de equipos ágiles.

Del *backlog* del programa, que contiene los requisitos del programa, se extraen las historias de usuario, descomponiendo esos elementos de mayor nivel en elementos más disgregados, que van a los *backlogs* de producto de cada uno de los equipos que conforman el programa. Esta labor se hace en las sesiones de planificación de incrementos de programa, que se realizan al inicio de cada PI. Es importante tener en cuenta las dependencias entre elementos que están desarrollando distintos equipos, para planificar de acuerdo con ellas. Cuando se determina una dependencia cruzada, es necesario que el elemento responsabilidad del equipo A esté planificado en una iteración anterior al del equipo B. Estas y otras consideraciones forman parte de las sesiones de planificación del PI.

Los equipos construyen sus funcionalidades y se integran continuamente, realizando al final de cada iteración demos individuales del equipo en cuestión seguidas de demos del sistema, las cuales serán conjuntas con los demás equipos y mostrarán el incremento del sistema integrado.

En cada equipo se encuentran los roles ya conocidos: el equipo de desarrollo, el facilitador o *scrum master* y el representante del cliente o *product owner*. A estos habrá que añadir otros roles para realizar la gestión a nivel de programa:

- Un *chief scrum master* o facilitador jefe, que gestione los procesos ágiles a nivel de programa.
- Un *product manager* o gestor del producto, que sea el dueño del *backlog* del programa, definiendo y priorizando las funcionalidades.
- Un arquitecto que guíe y armonice a los equipos en cuestiones de arquitectura.

- El equipo de sistema, que facilite la integración entre todos los equipos.
- Y los propietarios de negocio, las principales partes interesadas en el programa completo.

Los *scrum masters* de todos los equipos junto con el *chief scrum master* tendrán reuniones periódicas, típicamente semanales, donde compartirán los avances y los impedimentos encontrados. De igual manera, los *product owners* se reunirán con el *product manager* para compartir información sobre el alcance y los ajustes de priorización realizados. La coordinación es fundamental, puesto que la velocidad de todo el programa será irremediablemente la del equipo más lento.

5 DevOps

Muchas compañías ágiles están invirtiendo hoy día en DevOps, un paradigma relativamente moderno al que se hace referencia por primera vez en conferencias en 2008-2009. DevOps tiene grandes afinidades con los enfoques ágiles y con Lean, y por estos motivos se dedica un apartado en este tema a la explicación de sus conceptos y fundamentos.

DevOps no es una metodología, sino más bien una cultura. La visión tradicional de una organización de TI distribuye claramente funciones entre Desarrollo (Dev) y Operaciones (Ops). El área de desarrollo crea los productos, y el área de operaciones lo gestiona después de su puesta en producción. Los fieles a DevOps afirman que tratar las dos áreas como silos ha producido grandes perjuicios a los servicios.

DevOps extiende los principios ágiles, que proclaman la colaboración cercana entre equipos, al servicio completo. Se puede definir como la práctica por la cual los equipos de desarrollo y operaciones participan conjuntamente en todo el ciclo de vida del producto, desde su diseño, a su implementación y a su soporte. Entre sus ventajas, se encuentran:

- Los cambios se ponen en producción antes
- Los recursos se comparten con facilidad
- Los grandes sistemas son mejor gestionados y mantenidos

5.1 Valores y principios

DevOps no tiene una lista única de valores y principios como puede ocurrir con *Agile* y su manifiesto o con *Lean*. Sin embargo, hay modelos que se referencian con frecuencia cuando se definen los fundamentos de DevOps.

El modelo de valores más citado se denomina CAMS: Cultura, Automatización, Medición e Intercambio (Sharing).

- Cultura: La cultura se guía por el comportamiento. Desarrollo y operaciones han sido dos silos separados desde hace muchos años, cada uno con sus responsabilidades, objetivos, lenguaje y entendimiento. Sólo cambiando estos comportamientos y asunciones es como se puede impulsar el cambio en la cultura organizativa.
- Automatización: La automatización es una parte fundamental de DevOps. Una vez se comprende la cultura, se puede crear una capa de automatización para controlar los sistemas y las aplicaciones.
- Medición: Las métricas informan de lo que está ocurriendo y permiten hacer un seguimiento de los cambios realizados. Se debe medir los costes, ingresos y satisfacción del personal, entre otros.
- Intercambio: Compartir ideas e impedimentos está en la base de la colaboración. DevOps propone un ambiente colaborativo, transparente y abierto.

Por su parte, los principios más referenciados se conocen como *las tres vías*:

- Pensar en el sistema: Se debe comprender el sistema completo para poder optimizarlo. De nada sirve optimizar una parte si el conjunto no funciona con eficacia.
- Amplificar los ciclos de feedback: Se trata de crear y amplificar los ciclos de feedback entre todas las partes de la organización que formen parte de la cadena de valor, para obtener desarrollos y operaciones más productivos.
- Cultura de experimentación y aprendizaje: El foco se pone en realizar cosas nuevas, no en hablar de ellas sin ponerlas en práctica. Las habilidades se perfeccionarán haciendo, explorando y repitiendo.

5.2 Ciclo

Las etapas que comprende DevOps son las reflejadas en el siguiente gráfico, todas se realizan por un mix de recursos de TI no separados por departamentos, y además se soportan por medio de herramientas que automatizan los procedimientos:

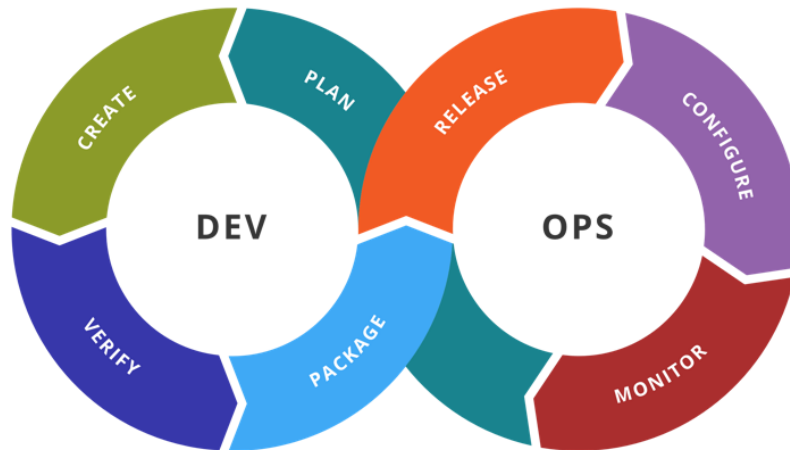


Figura 8 Etapas de DevOps.

- Planificar: En esta actividad se definen los requisitos de la aplicación y el valor para el negocio
- Crear: En esta etapa se diseña y se construye la solución.
- Verificar: Supone asegurar la calidad del producto desarrollado, por medio de las diferentes fases de pruebas.
- Empaquetar: El sistema está en preproducción y las tareas relacionadas con los pasos previos a la subida a producción se incluyen en esta fase.
- Entregar: Es decir, planificar y subir el software a producción.
- Configurar: Una vez que el sistema está en el entorno de producción, se deben completar la configuración completa.
- Monitorizar: Permite a la organización de TI medir el rendimiento de la infraestructura, el impacto en los usuarios finales, además de la producción de otras métricas y estadísticas. La información resultante de las actividades de monitorización impacta en las actividades de planificación nuevamente, en la medida en que surgen nuevas necesidades o se requieren cambios.

6 Lean

Lean en su esencia es un proceso sistemático para eliminar desechos o desperdicios en los procesos, entendiendo como tal los elementos que no aportan valor, y que tiene su origen en el mundo de la fabricación, en concreto se implantó originalmente en la cadena de producción de Toyota en los años 70 del siglo pasado. Ya en los primeros años de este siglo es cuando se han adaptado esas técnicas al desarrollo de sistemas.

6.1 Principios fundamentales

La metodología *Lean* para el desarrollo de software tiene como base 7 principios fundamentales:

- Eliminar el desperdicio: Las actividades de valor son aquellas por las que el cliente está dispuesto a pagar, el resto son desperdicio.
- Amplificar el aprendizaje: El conocimiento es importante, pues permite que el equipo sea más productivo y flexible, y se han de tomar acciones encaminadas a propagarlo y reforzarlo.
- Decidir tan tarde como sea posible: Este principio implica tomar las decisiones en el momento más avanzado en que se puedan tomar sin causar problemas. En áreas complejas o muy variables, esto es especialmente importante.
- Entregar tan rápido como sea posible: Entregar el producto mínimo viable para el cliente para a partir de ahí y, en base a su feedback, crear el producto final. Esta debe ser la forma de trabajo, en lugar de tratar de pensar desde el inicio todos los elementos que constituirán el producto perfecto y no entregar nada hasta tenerlo construido completamente.
- Empoderar al equipo: Escuchar a todos, proporcionarles las herramientas necesarias para hacer su trabajo e incentivar que generen ideas y tomen decisiones en su ámbito de actuación.
- Embeber la calidad: Los problemas de calidad generan desperdicio, en forma de retrasos, resolución y gestión de defectos, etc. Para evitarlo, lo mejor es adquirir prácticas que aseguren la calidad lo antes posible durante el proceso de construcción.
- Ver la totalidad: Una organización *Lean* trata de optimizar la cadena completa de valor, y no partes o equipos individuales.

Se puede ver la relación que tienen las filosofías ágiles con *Lean* en múltiples puntos, como el hecho de detallar los requisitos justo antes de construirlos (decidir tan tarde como sea posible), realizar el trabajo y la revisión en parejas (embeber la calidad), llevar a cabo sesiones diarias y retrospectivas (amplificar el aprendizaje), construir de forma iterativa (entregar tan rápido como sea posible), crear equipos auto-organizados (empoderar al equipo), reducir la burocracia (eliminar el desperdicio) o crear equipos multidisciplinares que puedan encargarse de una petición de principio a fin (ver la totalidad).

Lean implica la transformación de la organización, comenzando precisamente por la adopción de sus principios. Una forma visual de representar en un mismo diagrama los elementos característicos de *Lean* es lo que se conoce como la casa de *Lean*. En el tejado se representa el valor, que es el objetivo a alcanzar, mientras que los cimientos se fundamentan en el liderazgo. Los pilares de la casa lo conforman el respeto por las personas y su cultura, el flujo de trabajo, la innovación y la mejora continua.

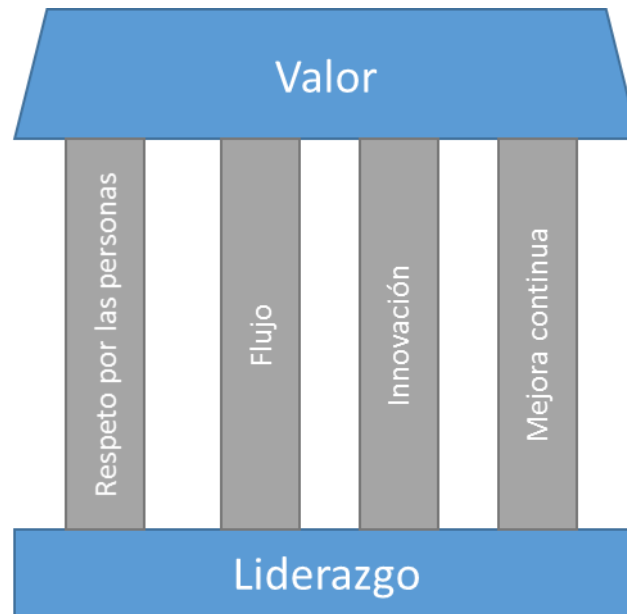


Figura 9 Casa de Lean.

- Respeto por las personas: Las personas son las que realizan todo el trabajo y se debe evitar sobrecargarles, forzarles a realizar trabajo que luego se desecha o imponerles deseos irrealizables. Se deben construir relaciones a largo plazo, basadas en la confianza.
- Flujo: Se debe optimizar la entrega de valor continua y sostenible, evitando retrasos por inicios-paradas-reinicios del proyecto, con alta calidad (el flujo sostenido depende de ello), gestionando la variabilidad, integrando con frecuencia y tomando decisiones informadas con feedback frecuente.
- Innovación: La innovación viene del que produce, y la validan los clientes. Se debe proporcionar tiempo y espacio para la creatividad y aplicar la innovación.
- Mejora continua: Se ha de optimizar el todo, considerando los hechos con atención y actuando rápidamente. Los que se adaptan más rápido, vencen.
- Liderazgo: Se debe liderar el cambio, desarrollando a las personas, enfatizando el aprendizaje a largo plazo, inspirando y alineándose con la misión de la organización y descentralizando las decisiones.
- Valor: El propósito es lograr el menor tiempo de entrega (*lead time*) que sea sostenible mantener en el tiempo, con la máxima calidad y el mayor valor entregado, así como el menor coste, con una alta motivación, seguridad y satisfacción del cliente.

La filosofía fundamental de *Lean* pasa por identificar las actividades que se realizan y que añaden valor al producto que se está construyendo, y las actividades que no. A las actividades que no añaden valor se les denomina desperdicio. Originalmente se definieron tres tipos de desperdicios, nombrados con palabras japonesas: *Muda*, que comprende el trabajo que absorbe recursos, pero no aporta valor; *Muri*, que indica el trabajo irrazonable impuesto sobre los trabajadores; y *Mura*, que incluye el trabajo que llega de forma irregular en lugar de llegar con un flujo constante. Siguiendo estas ideas, en los proyectos de desarrollo de software se han identificado siete tipos de desperdicios:

- El trabajo parcialmente hecho y que por tanto no se puede entregar.
- Proveer más funcionalidad de la solicitada.
- No usar el conocimiento de los miembros del equipo y tratar de reinventar la rueda.
- El cambio de manos, pasar el trabajo de unos a otros y el tiempo que esto requiere.
- Los retrasos, que afectan a los tiempos de entrega o de inicio de las actividades.
- Cambio de tareas de un miembro del equipo sin completar las que está haciendo.

- Defectos, funcionalidad errónea que no produce los resultados esperados.

6.2 Técnicas y prácticas

La metodología *Lean* adaptada al desarrollo de software no define un proceso concreto. Lo importante es demostrar que los procesos están alineados con los principios y valores antes mencionados, en este caso el proceso se puede decir que será *Lean*.

De igual manera no existen una serie de técnicas prescritas para *Lean*. Sin embargo, existen una serie de prácticas y técnicas que se adoptan habitualmente. En esta sección se van a repasar las más importantes.

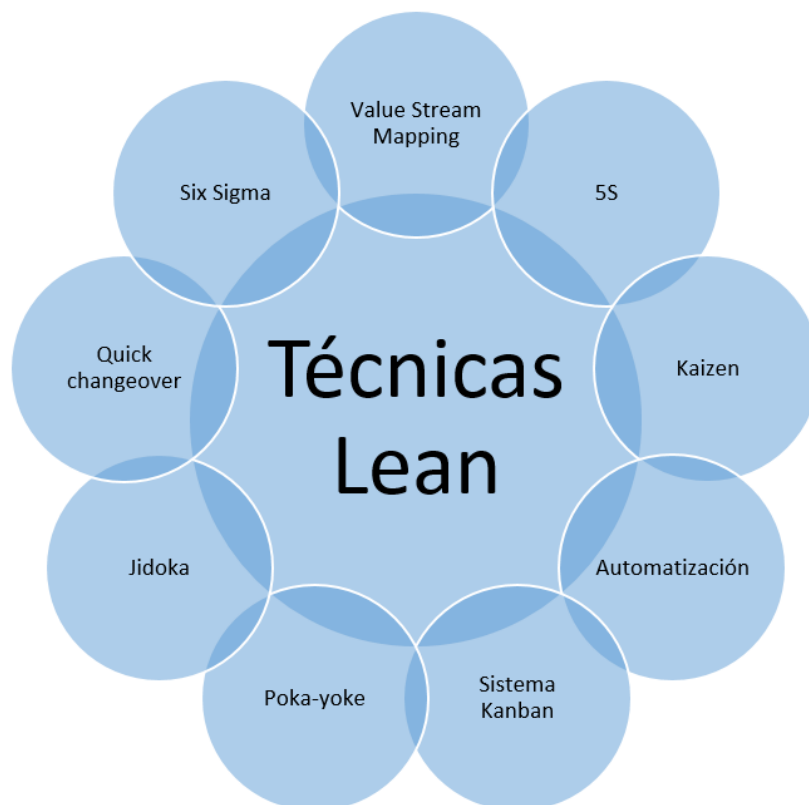


Figura 10 Técnicas aplicadas en *Lean*.

6.2.1 Value Stream Mapping (VSM)

El value stream mapping o mapa de flujo de valor es una herramienta de *Lean* utilizada para analizar el flujo de recursos y de información que se requiere desde que se solicita un producto hasta que se entrega al cliente.

Las cadenas de valor comienzan y acaban con un cliente: desde que éste realiza una petición hasta que se pone a su disposición. Representan la secuencia de actividades que se llevan a cabo, el promedio de su duración y las esperas entre cada una de ellas, identificando el tiempo empleado en dar valor al producto final y el tiempo dedicado a trabajo que no añade valor.

Su utilidad, más allá de la representación de la situación actual, es el diagnóstico de la misma para localizar puntos de mejora, como reducir retrasos o minimizar el retrabajo sobre cosas ya realizadas.

Una vez localizados todos los focos de mejora, se deben plasmar en un mapa de la cadena de valor futuro que ayude a la implementación de la nueva solución y que deberá ser actualizado con frecuencia siguiendo el principio de mejora continua.

El siguiente ejemplo muestra una VSM para un proceso con una eficiencia por debajo del 30% (4 horas aportan valor frente a 14 que no). Los tiempos de espera son los desperdicios que se pueden minimizar entregando el mismo valor al cliente.

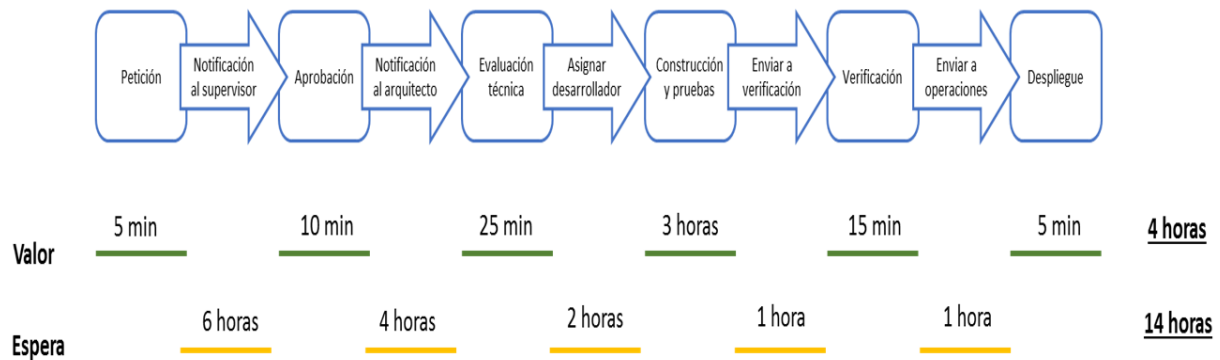


Figura 11 Ejemplo de mapa de flujo de valor.

6.2.2 5S

5S es una herramienta que ayuda a organizar el lugar de trabajo y a conseguir procesos más eficientes. En su aplicación a la ingeniería del software, el lugar de trabajo se extiende del lugar físico al ámbito lógico. Consta de los siguientes elementos:

- Organizar (Sort): Asegurarse de que cada elemento está en su lugar y, si no se necesita, eliminarlo. Servidores y equipos de trabajo no deben tener versiones obsoletas o en desuso, la documentación debe estar actualizada, etc.
- Ordenar (Set in Order): Ordenar los materiales para que sean fáciles de encontrar cuando se necesiten. La estructura de carpeta y organización de los ficheros debe seguir una lógica que facilite su localización, especialmente aquellos espacios de trabajo compartidos, etc.
- Limpiar (Shine): Mantener el lugar de trabajo limpio. Eliminar del código aquello que lo deslucen, como la falta de documentación, avisos en el código, etc.
- Estandarizar (Standardize): Establecer procedimientos formales para que todos los pasos se realicen correctamente.
- Sostener (Sustain): Continuar la estandarización a través de la formación y la comunicación.

6.2.3 Kaizen

El término Kaizen hace referencia a la mejora continua. Un evento Kaizen es el acto de realizar cambios a un proceso para que éste mejore. La esencia parte de considerar que deben ocurrir conversaciones sobre los problemas, en grupos de personas provenientes de distintos equipos y con diferentes perspectivas y conocimientos.

Dado un problema bien definido, los eventos Kaizen son una herramienta que reúne en un equipo a los representantes de las distintas áreas involucradas. Durante un espacio de tiempo de no más de una semana, trabajan en realizar las modificaciones necesarias en los procesos para solucionarlo. Tras el evento Kaizen, el equipo se disuelve y sus miembros vuelven a realizar sus labores habituales, y el proceso mejorado queda implantado.

6.2.4 Automatización

Como parte de la búsqueda de la alta calidad y la reducción de fallos, la automatización juega en *Lean* un papel importante. El uso de pruebas automatizadas, despliegues automáticos y factorías de software que automaticen el despliegue de los patrones de diseño y fomenten la creación de código estable son habituales en los procesos *Lean*.

6.2.5 Sistema Kanban

Kanban es la práctica que de forma simple y visual controla el flujo de estados de las tareas de un proceso. Permite limitar la cantidad de trabajo sin terminar (WIP, o work-in-progress) en cada una de las etapas del flujo de trabajo. Esto origina un efecto arrastre, puesto que no permite que se inicie una nueva tarea a menos que las que se están desarrollando por delante vayan avanzando en su estado. Si algún miembro del equipo se queda libre y en la primera etapa del proceso ya están el número máximo de elementos prefijado, antes de comenzar una nueva tarea ayudará a avanzar las tareas de otros miembros del equipo.

Hoy día los sistemas de Kanban suelen ser virtuales en lugar de usarse tarjetas físicas, aunque siguen mostrando la información de forma visual.

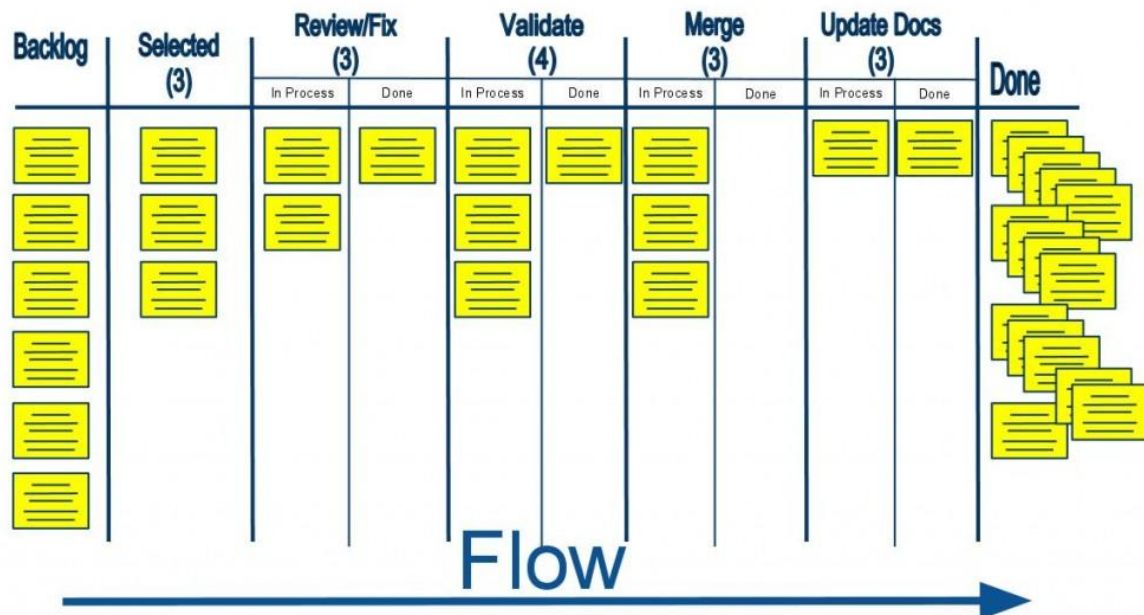


Figura 12 Ejemplo de visualización de un sistema Kanban.

6.2.6 Poka-yoke

Poka-yoke significa a prueba de errores. Es una técnica de calidad que se aplica con el fin de evitar errores en la operación de un sistema. Lo que se busca con esta forma de diseñar los procesos es eliminar o evitar equivocaciones ya sean de ámbito humano o automatizado.

Este sistema se puede implantar también para facilitar la detección de errores.

Entre las ventajas usar un sistema Poka-Yoke destacan las siguientes:

- Se reduce el riesgo de cometer errores en las actividades repetitivas.
- El trabajador puede centrarse en las operaciones que añadan más valor, en lugar de dedicar su esfuerzo a comprobaciones para la prevención de errores o a la subsanación de los mismos.

- Se mejora la calidad en su origen, actuando sobre la fuente del defecto, en lugar de tener que realizar correcciones, reparaciones y controles de calidad posteriores.
- Se caracterizan por ser soluciones simples de implantar y baratas.

Un ejemplo de un sistema a prueba de errores es aquel sistema de gestión que impide que un usuario modifique las tareas que están cogidas por otro miembro del equipo.

6.2.7 Jidoka

El concepto de Poka-yoke tiene como misión apoyar al trabajador en sus actividades rutinarias. En el caso en que el dispositivo forme parte del funcionamiento de una máquina, es decir, que sea la máquina la que realiza las tareas, estaremos hablando de otro concepto similar: Jidoka. Si existe una anomalía durante el proceso, este se detendrá ya sea automática o manualmente, impidiendo que los defectos avancen en el proceso.

Un ejemplo de Jidoka lo constituye la creación de un sistema de detección y generación de alarmas en procesos automáticos.

6.2.8 Quick changeover

En *Lean*, como en *Agile*, se realiza el producto sumando pequeños incrementos y tareas. El tiempo de cambio entre tareas es lo que se pretende agilizar con la técnica de quick changeover. Si esos tiempos son prolongados, se pierde productividad y se aumenta el desperdicio. Para conseguir un cambio rápido, se aplican una serie de técnicas:

- Realizar actividades en paralelo.
- Estandarizar. Las tareas similares que se realizan de forma parecida son mucho más fáciles de intercambiar.
- Eliminar operaciones no esenciales.
- Realizar una configuración previa que permita asegurar que se tienen todos los elementos necesarios para comenzar la tarea.
- Simplificar el código y las herramientas utilizadas.
- Medir para poder mejorar.

6.2.9 Six Sigma

Cuando la causa del problema es la variabilidad, Six Sigma presenta las claves que pueden ayudar a resolverlo. Es una metodología de mejora de procesos creada por Motorola para resolver problemas de calidad y productividad en la entrega de un producto o un servicio a un cliente.

Se centra en tres elementos: defectos, variabilidad y el cliente. Su meta se suele representar en la forma de una cifra: obtener 3,4 defectos por cada millón de oportunidades de que ocurra un defecto. La clave para controlar los defectos es controlar la variabilidad. Six Sigma se basa en la idea de que si la variabilidad se puede reducir hay menos oportunidades de que ocurran defectos.

Consta de cinco pasos:

- Definir: Determinar qué es el proyecto y qué se pretende conseguir.
- Medir: Se debe medir para conocer el proceso, controlarlo y poder mejorarlo. Hay que definir qué se quiere medir, y cómo medirlo.
- Analizar: Buscar las causas últimas de los problemas, qué está causando la variabilidad, qué está causando los defectos.

- **Mejorar:** Es la parte central de la metodología, se trata de implementar los cambios que resuelvan los problemas identificados y mejoren el rendimiento medido.
- **Controlar:** Es un paso muy importante, que permite poner en marcha el proceso de mejora continua en la organización. Se ponen en funcionamiento procedimientos y herramientas para asegurar que la solución es efectiva y se realizan cambios para mejorar.

¿Y por qué el nombre de Six Sigma? Six Sigma utiliza herramientas estadísticas para la caracterización y el estudio de los procesos. Sigma (σ) es la dispersión o desviación típica que da una idea de la variabilidad en un proceso. El objetivo de la metodología es reducir ésta de modo que el proceso se encuentre siempre dentro de los límites de tolerancia del producto, establecidos por los requisitos del cliente. Se puede clasificar la eficiencia de un proceso en función de su nivel de sigma, desde 1 sigma (690.000 defectos por millón de oportunidades) hasta 6 sigma (3,4 defectos por cada millón de oportunidades).

RESUMEN ESQUEMÁTICO

- **Introducción**

- Presentación de los contenidos del tema

- **Fundamentos de la gestión ágil de proyectos**

- Definición de agilidad: entrega iterativa y receptiva a los cambios durante el proceso
- Diferencias con metodologías tradicionales
- Manifiesto ágil: sustento común de todas las implementaciones ágiles
 - Valores: foco en las personas y sus interacciones
 - Principios
- Cuándo elegir metodologías ágiles para gestionar un proyecto
 - Características del proyecto que lo hacen candidato
- Metodologías ágiles más populares
 - Scrum: la más popular
 - Kanban: gestión del flujo de trabajo
 - XP: programación por parejas, ciclos cortos
 - FDD: el proceso se guía por funcionalidades
 - TDD: los requisitos se transforman en guiones de pruebas antes de empezar
 - BDD: los requisitos se definen con un lenguaje natural estructurado
 - DSDM: centrada en la necesidad y el valor para el negocio
 - Crystal: una de las más ligeras y adaptables
 - Lean: minimizar el desperdicio y el tiempo de entrega
 - También mixtas, como Scrumban o Scrum+XP
- Situación de agile en las organizaciones
 - Se usa en el 94% de las organizaciones, aunque por menos de la mitad de los equipos
 - El 80% de las organizaciones está en un nivel bajo de maduración
 - Scrum es la metodología más utilizada: 58% o casi 80% si contamos las versiones híbridas
 - El beneficio más apreciado es la capacidad para gestionar prioridades cambiantes
 - Su uso ya es común en las administraciones públicas (caso real: SOROLLA2)

- **Ciclo de vida de un proyecto ágil**

- Concebir: se determina qué se quiere hacer y cómo se va a lograr
 - Conformación del equipo: equipo de desarrollo, facilitador y representante del cliente
 - Duración de las iteraciones: entre 1 y 10 semanas, ajustar en función de elementos del cliente, del equipo y del proyecto.
 - Herramientas de gestión: Jira, Excel, Microsoft TFS y VersionOne son las principales
- Especular: para marcar la dirección y el objetivo, identificando las funcionalidades, estimándolas y distribuyéndolas

- Registro de requisitos: backlog del producto y backlog de la iteración
- Distribución de funcionalidades por iteraciones: priorización en base al valor, el riesgo y el esfuerzo
- Explorar: programación y pruebas, reuniones diarias del equipo
 - Control del avance: gráficos de burndown (a nivel de proyecto y de iteración) y burn-up
- Adaptar: ver resultados, recoger feedback y adaptarse
 - Sesiones de demo junto con todas las partes interesadas
 - Sesiones retrospectivas del equipo
 - Solución de problemas: gestión distribuida, casos típicos
- Cerrar: cuando se terminan las funcionalidades, el tiempo o el presupuesto
 - Tareas administrativas y momento de comunicar los resultados
 - Documentar lecciones aprendidas
 - Repasar elementos que han quedado pendiente
 - Seguimiento y evaluación de los resultados
- **Agile a gran escala**
 - Para programas grandes
 - Necesidad de coordinación y roles específicos a nivel de programa
 - SAFe y Scrum de scrums como implementaciones más seguidas
 - En cada equipo se mantienen los mismos principios y organización explicados, que se escalan para obtener un equipo de equipos ágiles
 - Coordinación: sincronización y cadencia entre los equipos
 - Incrementos de programa o PI cada un número determinado de iteraciones (típicamente 10 semanas)
- **DevOps**
 - Paradigma que pretende acabar con la separación entre desarrollo y operaciones
 - Valores y principios
 - Ciclo: planificar – crear – verificar – empaquetar – entregar – configurar – monitorizar
- **Lean**
 - Proceso sistemático para eliminar desperdicio en el proceso (lo que no aporta valor al cliente), aplicado a ingeniería de software
 - Principios
 - Casa de Lean
 - Siete tipos de desperdicios
 - Técnicas y prácticas
 - VSM: mapa de flujo de valor que muestra la secuencia de actividades con detalle del tiempo que aporta valor y los tiempos de espera o que no aportan valor
 - 5S: herramienta que ayuda a organizar el lugar de trabajo y a conseguir procesos más eficientes (Sort – Set in order – Shine – Standardize – Sustain)

- Kaizen: eventos para la mejora continua
- Automatización: como parte de la búsqueda de la calidad y reducción de fallos
- Sistema Kanban: control del flujo de estado de las tareas
- Poka-yoke para crear sistemas a prueba de errores
- Jidoka: como el anterior, pero en caso de que el proceso lo realice una máquina
- Quick changeover: tiempo de cambio entre tareas
- Six sigma: para reducir los defectos a través de una reducción de la variabilidad

GLOSARIO

APM: Agile Project Management
DSDM: Dynamic Systems Development Method
FDD: Feature Driven Development
PI: Product Increment
SAFe: Scaled Agile Framework
SaaS: Software as a Service
TDD: Test Driven Development
TDS: Team Foundation Server
TI: Tecnologías de la Información
VSM: Value Stream Mapping
WIP: Work in progress
XP: Extreme Programming

PREGUNTAS DE TEST

- 1) ¿Cuál es un ejemplo de gestión del cambio en las prácticas ágiles?
 - a) El alcance está bloqueado lo que dura el proyecto, no puede haber cambios
 - b) Las propuestas de cambio se documentan y pasan por un comité de aprobación
 - c) En base a los comentarios del usuario en las sesiones de demo, se añaden nuevos elementos al backlog de forma ordenada
 - d) Ninguna de las tres.
- 2) Los equipos ágiles...
 - a) No tienen horario de trabajo fijo
 - b) Trabajan mejor de forma deslocalizada
 - c) Precisan ser microgestionados
 - d) Son multidisciplinares
- 3) En los proyectos ágiles...
 - a) Se entrega valor rápidamente
 - b) Se entrega valor una sola vez
 - c) La entrega de valor es secundaria
 - d) El valor no se puede medir
- 4) Para tener éxito en un proyecto ágil, la colaboración...
 - a) Es un elemento clave
 - b) Se incentiva con herramientas colaborativas
 - c) Todo el equipo colabora como si fuera uno, tanto la parte del cliente como el equipo de desarrollo
 - d) Todas son ciertas
- 5) La gestión ágil de proyectos es el proceso por el cual los proyectos se implementan...
 - a) En pequeñas partes
 - b) En grandes partes
 - c) En dos o cuatro semanas
 - d) En dos o cuatro fases
- 6) ¿Por qué fases atraviesa un proyecto ágil?
 - a) Concepción, Especulación, Exploración, Adaptación y Cierre
 - b) Inicio, Ejecución y Cierre
 - c) No hay fases definidas
 - d) Análisis, Plan, Diseño, Desarrollo, Pruebas e Implantación
- 7) ¿Qué tipo de proyectos son buenos candidatos para ser gestionados de forma ágil?
 - a) Los proyectos largos y complejos
 - b) Proyectos que requieren entregas rápidas y pueden ser entregados por partes
 - c) Proyectos que ya se han hecho antes y son repetibles
 - d) Proyectos con equipos internacionales
- 8) En un proyecto ágil, el cliente junto con el equipo de desarrollo...

- a) Cambia el alcance en el transcurso de una iteración
 - b) Redefine el alcance del proyecto de forma constante
 - c) Reprioriza para determinar las funcionalidades a incluir en la siguiente iteración
 - d) Celebra los éxitos en la reunión semanal
- 9) La palabra que mejor describe el trabajo del gestor de proyectos ágiles es:
- a) Controlador
 - b) Facilitador
 - c) Organizador
 - d) Comunicador
- 10) Al cierre de un proyecto:
- a) Se revisa el valor de las funcionalidades que han quedado pendientes y si es necesario se propone un nuevo proyecto
 - b) Se realiza una iteración adicional (iteración cero) de duración flexible para completar todos los elementos del *backlog*
 - c) El equipo de proyecto ha de responder por las funcionalidades no realizadas, aplicándose las penalizaciones acordadas
 - d) Nunca quedan funcionalidades pendientes
- 11) En las prácticas ágiles:
- a) Lo más importante es el plan
 - b) Es más importante tener al equipo motivado que ajustarse al plan
 - c) Es más importante la respuesta al cambio que ajustarse al plan
 - d) Es más importante cumplir el horario de trabajo que ajustarse al plan
- 12) Una historia de usuario:
- a) Es un elemento de tamaño procesable que se añade a una iteración para ser desarrollado
 - b) Generalmente se escriben con un formato predefinido
 - c) Incluye un criterio de aceptación
 - d) Todas son ciertas
- 13) ¿Cómo se estima habitualmente en un proyecto ágil?
- a) Directamente en horas
 - b) De forma relativa: en puntos, o tamaños (XS, S, M, L, XL), o similar
 - c) No se estima, se desarrolla lo que se pueda en una iteración
 - d) No se estima, se asigna un esfuerzo a posteriori
- 14) ¿Cuáles de las siguientes son metodologías ágiles?
- a) Agile Manifesto, Jira, Scrum, XP
 - b) APM, Scrum, Métrica, Agile Delivery Methods (ADM)
 - c) XP, Crystal, FDD, DSDM
 - d) XP, BDD, Scrum, Slack
- 15) En las prácticas ágiles, la planificación es un compromiso
- a) No, la planificación siempre es una estimación o una intención, nada más que una especulación
 - b) Sí representa un compromiso cuando se planifica la iteración que va a comenzar
 - c) No se planifica, se es flexible al cambio

- d) Todas son falsas
- 16) La comunicación entre cliente y equipo de desarrollo ocurre:
 - a) Sólo en la fase de revisión de la iteración
 - b) Puede darse con frecuencia, por ejemplo diariamente
 - c) Sólo a través del representante del cliente
 - d) Sólo a través del gestor de proyecto
- 17) El progreso se mide:
 - a) Utilizando un gráfico de trabajo pendiente o *burndown chart*
 - b) El equipo rellena por sí mismo el avance sobre un diagrama de Gantt
 - c) El gestor de proyecto cuestiona a cada miembro
 - d) Una vez que finaliza una iteración
- 18) La velocidad de ejecución en *agile*:
 - a) Es impredecible
 - b) Determina la cantidad de elementos del *backlog* que el equipo puede completar en una iteración
 - c) Es un factor resultado de un acuerdo entre el equipo y el cliente
 - d) Mide las funcionalidades que se desarrollan por mes
- 19) Durante las sesiones de demo:
 - a) Se anima al cliente a que dé su opinión y solicite cambios si son necesarios
 - b) El cliente debe jugar un papel secundario y escuchar
 - c) La opinión del cliente es bienvenida, pero debe hacerse en una sesión separada
 - d) Participa sólo el equipo de desarrollo
- 20) Los requisitos son recogidos e introducidos en el *backlog*:
 - a) Una vez se ha finalizado una iteración
 - b) Por el representante del cliente, también llamado propietario del producto
 - c) Por el equipo de desarrollo
 - d) Por el usuario de negocio directamente
- 21) ¿Es posible realizar grandes proyectos con métodos ágiles?:
 - a) No, *agile* es para proyectos y equipos pequeños, donde la colaboración es vital
 - b) Sí, pero no está recomendado, para estos casos es preferible una metodología tradicional
 - c) Sí, aplicando metodologías como SAFe
 - d) Sí, aplicando metodologías como Extreme Programming
- 22) Cuando se aplica *Agile* a escala...
 - a) Los equipos ágiles trabajan de forma coordinada, con la misma cadencia, sincronizando las iteraciones y realizando integración continua
 - b) Cada equipo trabaja por su cuenta, y en un punto predefinido, llamado incremento de programa o PI, ponen sus avances en común
 - c) El programa se disgrega en proyectos que funcionan con autonomía para evitar distracciones
 - d) No existe una capa organizativa y de gestión a nivel de programa, puesto que todos los equipos se auto-organizan
- 23) ¿Qué es DevOps?

- a) Es una metodología ágil de gestión de proyectos
 - b) Es un paradigma de trabajo que pretende fusionar en sentido amplio los equipos tradicionales de Desarrollo y Operaciones
 - c) Es una cultura organizativa, que promueve el trabajo en equipo y las pruebas continuas para prevenir defectos
 - d) Es una herramienta de apoyo a la gestión de proyectos ágiles
- 24) ¿Cuál es la principal premisa de Lean?:
- a) Ser flexibles ante todas las circunstancias
 - b) Eliminar la burocracia
 - c) Eliminar todo aquello que no aporta valor
 - d) Colaboración entre todas las partes
- 25) ¿Cuáles de las siguientes son prácticas útiles para los desarrollos de software con *Lean*?
- a) Value Stream Metrics (VSM), Kanban, Automatización y Jidoka
 - b) Six Sigma, 6S, Rapid Evolution y Mapa de desperdicio
 - c) 6S, Poka-Yoke, Kanban y Kaizen
 - d) 5S, Value Stream Mapping (VSM), Kanban y Automatización
- 26) ¿Cuál de los siguientes no es un principio de *Lean*?
- a) Decidir tan pronto como sea posible
 - b) Amplificar el aprendizaje
 - c) Dar poder al equipo
 - d) Embeber la calidad
- 27) ¿Qué se busca cuando se hace un mapa del flujo de valor en *Lean*?
- a) Identificar el trabajo que no aporta valor
 - b) Mostrar el estado actual de un proceso
 - c) Mejorar el proceso, minimizando el desperdicio
 - d) Todas son ciertas
- 28) ¿Cuál es el propósito de las 5S aplicadas a la gestión de proyectos de sistemas?:
- a) Organizar el lugar de trabajo, tanto físico como lógico
 - b) Realizar un seguimiento de los objetivos
 - c) Cohesionar al equipo
 - d) Ninguna de las anteriores
- 29) En Kanban, ¿es posible tener más de 10 tareas abiertas?
- a) Sí, siempre que los miembros del equipo sean al menos 10
 - b) Sí, siempre que se desee
 - c) Sí, siempre que no se supere el umbral de WIP de cada etapa
 - d) No, siempre entre 5 y 10 como máximo
- 30) Si se pretende reducir el tiempo entre que llega una petición y se entrega el resultado al cliente, ¿qué metodología aplicarías?:
- a) Scrum
 - b) Lean
 - c) Scrumban

- d) Todas son opciones correctas
- 31) Las metodologías ágiles y Lean...
- a) Comparten muchas partes de su filosofía
 - b) Son contrapuestas
 - c) Definen lo mismo
 - d) Ninguna de las anteriores
- 32) Lean promueve entregar lo más rápido que sea posible
- a) Entregando un producto mínimo viable y mejorándolo iterativamente
 - b) Creando equipos fuertes y numerosos
 - c) Posponiendo las pruebas hasta el final
 - d) Todas son correctas
- 33) Entre los pilares de Lean, reflejados en las columnas de la llamada casa de Lean, no se encuentra
- a) Flujo
 - b) Valor
 - c) Innovación
 - d) Mejora continua
- 34) ¿Cuál es un ejemplo de desperdicio en Lean aplicado al desarrollo de software?
- a) El trabajo parcialmente hecho y que por tanto no se puede entregar
 - b) Proveer más funcionalidad de la solicitada
 - c) No usar el conocimiento de los miembros del equipo y tratar de reinventar la rueda
 - d) Todos los anteriores
- 35) Revisar el código por parejas, ¿de qué principio Lean sería mejor ejemplo?
- a) Ver la totalidad
 - b) Empoderar al equipo
 - c) Embeber la calidad
 - d) Entregar rápido

SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
1	c
2	d
3	a
4	d
5	a
6	a
7	b
8	c
9	b
10	a
11	c
12	d
13	b
14	c
15	b
16	b
17	a
18	b
19	a
20	b
21	c
22	a
23	b
24	c
25	d
26	a
27	d
28	a
29	c
30	d
31	a
32	a
33	b
34	d
35	c

BIBLIOGRAFÍA BÁSICA

1. Agile Manifesto, <http://agilemanifesto.org>
2. Annual State of Agile report, <http://stateofagile.versionone.com/>
3. Agile Project Management (Jim Highsmith)
4. Lean Software Development, an Agile Toolkit (Mary y Tom Poppendieck)

BIBLIOGRAFÍA PARA AMPLIAR EL TEMA

5. The art of Agile Practice (Bhuvan y Unhelkar)
6. Lean Product and Process Development (Allen Ward y Durward Sobeck)
7. The DevOps Handbook (Gene Kim, Patrick Debois, John Willis, John Allspaw, y Jez Humble)